Evolving Dataset-Specific Split Scores and Out-of-Bag Residual Correction for Tree Ensembles

Vu Gia Nam, Pham Vu Tuan Dat, Long Doan, Huynh Thi Thanh Binh

Abstract

Tree ensembles are strong tabular learners, yet their predictive behavior is largely determined by fixed, hand-designed split criteria such as Gini impurity or variance reduction, which impose a uniform inductive bias across heterogeneous datasets. We propose EvoGraft, a framework that evolves split scores and grafts out-of-bag residual corrections onto the resulting forest, improving tabular prediction at two complementary levels: tree construction and forest-level prediction. EvoGraft first evolves executable, dataset-specific split-scoring programs, evaluated by predictive fitness on held-out data, so that tree structures adapt to each dataset rather than to a universal impurity measure. After the forest is built, out-of-bag instances are grouped into buckets according to ensemble disagreement and leaf support; within each bucket, cross-validation on out-of-bag residuals selects the correction structure. The parameters of the selected correctors are fitted from out-of-bag residuals, and a disjoint calibration fold sets the strength of the correction. Experiments on fourteen tabular benchmarks spanning seven classification and seven regression tasks show that EvoGraft is competitive with tree-based and LLM-based baselines, obtaining the best mean rank on both classification and regression.

Keywords

Tree Ensembles, Evolutionary Computation, Large Language Models, Tabular Data

1 Introduction

Tabular data are common in many real-world applications, including healthcare [32], finance [9, 24], and manufacturing [6]. Although neural models for tabular learning have improved substantially [14], tree ensembles remain highly competitive on many tabular problems [15]. However, widely used methods still rely on fixed, hand-designed split criteria, such as Gini impurity or variance reduction, which apply the same local objective across datasets [4, 8, 20]. These criteria encode general-purpose preferences for partitioning data, while individual datasets differ in their feature and target distributions, noise levels, and feature–target relationships. The purpose of evolving a split-scoring program is therefore to adapt tree construction to the characteristics of a particular dataset, using predictive performance on development-validation data to guide the search. Prior work has tried to reduce this limitation through evolutionary algorithm design: HEAD-DT evolves complete decision-tree induction algorithms by selecting and combining their components [2], while ESC-GE uses grammatical evolution to generate split criteria from a predefined grammar [3]. These studies suggest that evolutionary search can replace fixed, hand-designed rules with data-adaptive tree construction, motivating our focus on evolving dataset-specific split-scoring programs for tree ensembles.

The emergence of large language models (LLMs) has introduced a new direction for machine learning, where LLMs are used not only as predictors but also to design and improve components of the learning pipeline. For tabular data, LLMs have been used as predictors [11, 17, 39], as feature engineers [16, 18], and to guide decision-tree induction [26, 36], while DeLTa [37] uses an LLM-generated rule to correct the errors of an existing tree-based predictor. These approaches motivate studying how dataset-specific tree construction and forest-level residual correction can be combined within a single framework.

To address this gap, we propose EvoGraft, a framework that adapts both tree construction and forest-level correction to each dataset. EvoGraft first evolves, with an LLM as the variation operator, an executable split-scoring program that is used throughout tree construction instead of relying on a fixed criterion. After the ensemble is built, it groups out-of-bag (OOB) samples by model uncertainty and, within each group, selects the structure of an OOB residual corrector by cross-validation over a small discrete catalog. Only this structure is selected; all numerical parameters and the final correction strength are fitted directly from data. Our main contributions are:

- We introduce a method that evolves an executable, dataset-specific split-scoring program and uses it throughout tree construction in place of a fixed criterion such as Gini impurity or variance reduction.
- We propose an uncertainty-aware OOB residual correction mechanism for the trained forest. EvoGraft uses out-of-bag predictions to identify systematic prediction errors in uncertainty buckets and selects the structure of each bucket's corrector by cross-validation on its out-of-bag residuals, while all numerical parameters are fitted from data.
- We evaluate EvoGraft against tree-based and LLM-based methods on 14 CARTE datasets, complemented by full-data and few-shot split-criterion comparisons on OpenML, cross-domain transfer, feature engineering, and component ablations. EvoGraft achieves the lowest mean rank in both CARTE tasks under the tested configurations.

2 Related Work

2.1 LLM for Tabular Data

LLMs for feature engineering. A growing line of work uses LLMs to improve the representation of tabular data before prediction. CAAFE [18] generates context-aware features from dataset descriptions and evaluates them with downstream models. FeatLLM [16] generates feature rules from a few labeled examples. These approaches improve the feature space while leaving prediction to a downstream model. EvoGraft addresses the unchanged downstream split objective by evolving a dataset-specific scoring program.

LLMs as zero-shot and few-shot learners. Another direction uses LLMs directly as learners for tabular data. LIFT [11] fine-tunes language models on inputs and outputs serialized as text. TabLLM [17] serializes rows into natural language for few-shot classification. TABLET [34] studies how task instructions help zero- and few-shot prediction. P2T [29] builds prompt demonstrations from heterogeneous source tables, and InsightTab [39] distills insights from training examples for few-shot classification. These methods rely on pretrained knowledge, task descriptions, and a limited number of examples. EvoGraft avoids inference-time LLM calls by deploying the evolved forest and data-fitted correctors.

LLMs for tree and forest construction. More recent work combines LLMs with decision trees and tree ensembles. LLM Trees [23] study zero-shot decision-tree induction and embedding, directly eliciting tree structures from LLMs. LLEGO [26] uses LLMs together with evolutionary search to guide decision-tree induction, allowing semantic information from the task to influence how trees are constructed. MAPLE [21] uses multiple LLM agents to construct tree ensembles, with different agents contributing to the tree-building process. FORESTLLM [36] combines LLM knowledge with random forests for few-shot tabular learning and uses LLM-derived information during tree splitting and leaf prediction. DeLTa [37] takes a different approach by starting from an existing tabular predictor and using an LLM-generated rule to correct its residual prediction errors. EvoGraft addresses both unchanged split objectives and residual errors by combining split-score evolution with OOB correction.

2.2 Automated Heuristic Design

Automated heuristic design (AHD) aims to automatically discover executable rules or algorithms for optimization problems. Recent work uses LLMs as program generators inside a search loop, where candidate heuristics are generated, evaluated, and improved using performance feedback. FunSearch [33] combines an LLM with evolutionary program search to repeatedly generate and select high-performing programs. Evolution of Heuristics (EoH) [25] further develops this idea by using LLMs with evolutionary operators to generate and refine heuristic algorithms. Several methods improve how the search process uses feedback. ReEvo [38] introduces reflective evolution, where the LLM compares better and worse heuristics and uses the resulting feedback to guide later generations. HSEvo [10] focuses on search diversity by combining LLM-based generation with harmony search and genetic algorithm mechanisms. MCTS-AHD [41] instead organizes heuristic generation with Monte Carlo Tree Search, providing a more structured balance between exploring new candidates and improving promising ones. HiFo-Prompt [7] further improves heuristic generation by using hindsight from previous solutions and foresight about the current search state to adjust the prompting strategy.

3 Methodology

Figure 1 presents the two stages of EvoGraft. The first stage evolves a dataset-specific split-scoring program using predictive fitness and symbolic experience extracted from the induced forests. The frozen best program is then used to construct an ensemble whose OOB evidence supports residual adaptation with cross-validated structure selection, data-driven parameter fitting, and held-out calibration in the second stage.

3.1 Problem Formulation

Problem Setup. Let $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$ denote a tabular dataset with N samples and p input features, where $\mathbf{x}_i \in \mathbb{R}^p$. For classification, $y_i \in \{1, \dots, C\}$ with C classes, while for regression, $y_i \in \mathbb{R}$. We seek to learn an ensemble of K decision trees, $\mathcal{F} = \{T_k\}_{k=1}^{K}$, that maps an input $\mathbf{x}$ to a prediction and generalizes to unseen data. Each decision tree [5] partitions the feature space into regions and assigns a prediction to each terminal leaf. For classification, each tree produces a class-probability vector $T_k(\mathbf{x}) \in [0,1]^C$, while for regression it produces a scalar prediction $T_k(\mathbf{x}) \in \mathbb{R}$. The ensemble prediction is obtained by aggregating predictions across its trees.

Bagging and OOB Evaluation. Each tree T_k is trained on a bootstrap sample $\mathcal{D}^{(k)}$ drawn with replacement from $\mathcal{D}$, while samples not selected in $\mathcal{D}^{(k)}$ form its out-of-bag (OOB) set O_k [4]. For each training sample i, let $\mathcal{K}_i = \{k \in \{1, \dots, K\} : i \in O_k\}$ denote the set of trees for which sample i is OOB. Its OOB prediction averages only these trees (a class-probability vector for classification, a scalar for regression). Because sample i is not used to train any tree in $\mathcal{K}_i$, OOB predictions estimate predictive behavior on unseen samples without an additional validation split.

3.2 Experience-Guided Split-Score Evolution

Consider an internal node v containing the training samples that reach that node. For each candidate split $s \in \mathcal{S}_v$, let $\mathbf{y}_{v,s}^{L}$ and $\mathbf{y}_{v,s}^{R}$ denote the target values assigned to the left and right child, respectively. Instead of ranking candidates using a fixed impurity criterion, EvoGraft represents the split criterion itself as an executable program g and defines

$$S_g(v,s) = g\Big(\mathbf{y}_{v,s}^{L}, \mathbf{y}_{v,s}^{R}, \mathbf{c}_{v,s}\Big), \qquad s_v^{*} = \arg\max_{s \in \mathcal{S}_v} S_g(v,s). \tag{1}$$

The same program g is applied throughout tree construction, while its behavior may vary across nodes and candidate splits through the context $\mathbf{c}_{v,s}$, which includes node depth and sample fraction, left and right child fractions and split balance, previous use of the candidate feature, features appearing on the root-to-parent path, and the number of input features or classes. The context contains no validation or test labels, predictions, or fitness values, so g ranks each candidate using only information available during tree construction.

A candidate split-scoring program is evaluated through the predictive performance of the tree structures it induces rather than through the quality of individual splits in isolation. Let $\mathcal{M}$ denote a set of development runs. For each run $\ell \in \mathcal{M}$, a model $\mathcal{F}_g^{\ell}$ is trained using g to rank candidate splits throughout tree construction, and its predictive quality is measured on the corresponding development data $\mathcal{D}_{\text{dev}}^{\ell}$. We define

$$\mathcal{J}(g) = \frac{1}{|\mathcal{M}|} \sum_{\ell \in \mathcal{M}} Q\Big(\mathcal{F}_g^{\ell}, \mathcal{D}_{\text{dev}}^{\ell}\Big), \qquad \widetilde{\mathcal{J}}(g) = \rho\, \mathcal{J}(g), \tag{2}$$

where Q is the task-specific evaluation metric. We set $\rho = 1$ for metrics to maximize, such as accuracy, AUROC, and R^2, and $\rho = -1$ for

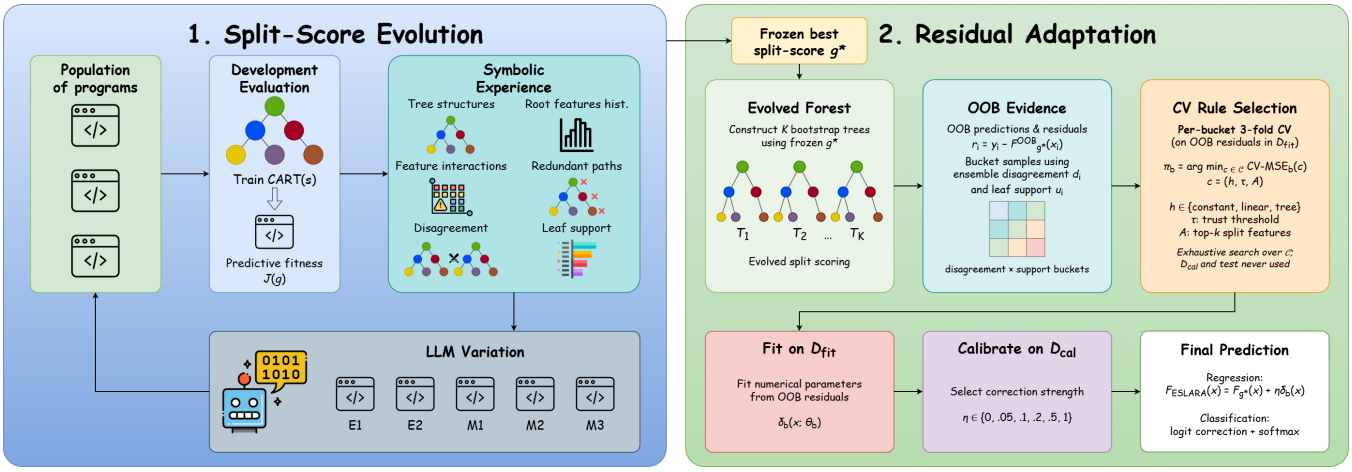


Figure 1: Overview of EvoGraft. Split-score evolution uses predictive fitness and symbolic forest experience to evolve a dataset-specific scoring program $g^\star$. The resulting forest provides OOB residual and uncertainty evidence from which cross-validation selects correction structures; their numerical parameters are fitted from data and their strength is calibrated before final prediction.

metrics to minimize, such as log-loss, MAE, and RMSE. Therefore, larger values of $\widetilde{\mathcal{J}}(g)$ are always preferred during evolution.

Initialize and evaluate. Let P and E denote the population and elite-pool sizes. The initial population combines reference splitting criteria with randomly generated programs. Each individual pairs a natural-language rationale with an executable `score_split` function. Valid programs are evaluated by retraining the development models, ranked by $\widetilde{\mathcal{J}}$, and truncated to the top P unique individuals.

Extract symbolic experience. For each of the top E individuals, EvoGraft retrains its development model and distills the induced forest into a textual experience report. Each report contains (i) meta-information about the task, including the feature inventory with numeric and categorical counts, the number of trees, and the training and validation sizes; (ii) forest-level diagnostics: the histogram of root-split features, which exposes dominant and under-explored features, the most frequent feature-interaction pairs co-occurring along root-to-leaf paths, clusters of redundant paths grouped by the Jaccard similarity of their validation coverage, and the validation samples with the highest cross-tree prediction disagreement, listed together with the leaf rules that produced each prediction; and (iii) the best and worst representative trees, serialized with their per-split scores and node diagnostics (depth, leaf count, feature reuse along branches, and leaf support skew), accompanied by derived recommendations on which behaviors to prefer or avoid. All statistics are computed exclusively from development training and validation data.

Select parents and generate offspring. Parents are sampled with replacement from the ranked population using

$$p_r = \frac{(P + r + 1)^{-1}}{\sum_{j=0}^{P-1}(P + j + 1)^{-1}}, \tag{3}$$

where $r \in \{0, 1, \ldots, P-1\}$ is the rank of an individual in the population sorted in descending order of $\widetilde{\mathcal{J}}$. The recombination operators E1 and E2 each sample several parents, whereas the mutation operators M1–M3 act on a single parent. E1 explores globally by requesting a heuristic with a substantially different mathematical form from its parents. E2 recombines parents by preserving their well-supported successful splits, altering the purity-versus-balance trade-off at weak nodes, and deriving a new form from observed failure patterns rather than copying any parent. M1 repairs failures by studying the reported validation errors, weak leaves, and symbolic tree paths to address one supported failure pattern while preserving successful behavior. M2 tunes constants by re-weighting the candidate-dependent purity, balance, and small-child penalty terms instead of rescaling the whole score. M3 simplifies by identifying which of purity gain, partition balance, or split information is over-emphasized and rebuilding the heuristic around a candidate-dependent trade-off. The symbolic experience extracted in the previous step accompanies every operator: all five prompts receive the parent programs, their fitness, and the elite experience text.

Validate and select survivors. Unsafe, invalid, duplicate, or non-finite offspring are rejected. Valid offspring are evaluated by retraining the development models. Parents and offspring are merged, ranked, deduplicated, and truncated to the best P programs, yielding elitist $(\mu + \lambda)$ selection.

The loop terminates at the generation or evaluation budget. The best program

$$g^\star = \arg\max_{g \in \mathcal{G}} \widetilde{\mathcal{J}}(g) \tag{4}$$

over the set $\mathcal{G}$ of evaluated programs is frozen and used for all subsequent forest construction.

3.3 OOB Residual Adaptation

The frozen score $g^\star$ determines *where* trees split; this second stage adapts the forest *after* fitting, using out-of-bag (OOB) residuals.

Forest construction. Each of the K trees is trained on an independent bootstrap replica of the training data, and every split of every tree is scored by the frozen program $g^\star$ instead of a fixed CART criterion. The evolved forest and its OOB counterpart are

$$\widehat{F}_{g^\star}(\mathbf{x}) = \frac{1}{K}\sum_{k=1}^{K} T_k(\mathbf{x}), \qquad \widehat{F}^{\mathrm{OOB}}_{g^\star}(\mathbf{x}_i) = \frac{1}{|\mathcal{K}_i|}\sum_{k\in\mathcal{K}_i} T_k(\mathbf{x}_i), \quad (5)$$

with $\mathcal{K}_i$ as defined above ($\widehat{F}_{g^\star}$ is used if $\mathcal{K}_i = \emptyset$).

Uncertainty-aware bucketing. Let $\mathcal{D}_{\mathrm{cor}}$ denote the OOB correction data. We define the OOB correction target as

$$\mathbf{r}_i = \begin{cases} y_i - \widehat{F}^{\mathrm{OOB}}_{g^\star}(\mathbf{x}_i), & \text{regression,} \\ \mathbf{e}_{y_i} - \widehat{\mathbf{p}}^{\mathrm{OOB}}_{g^\star,i}, & \text{classification,} \end{cases} \quad (6)$$

where $\widehat{\mathbf{p}}^{\mathrm{OOB}}_{g^\star,i} = \widehat{F}^{\mathrm{OOB}}_{g^\star}(\mathbf{x}_i)$ is the OOB class-probability vector and $\mathbf{e}_{y_i}$ is the one-hot encoding of class y_i; for classification, $\mathbf{r}_i$ is thus the negative gradient of the cross-entropy with respect to the logits, while for regression it is scalar-valued. Two uncertainty indicators are attached to every sample: the ensemble disagreement $d_i = \frac{1}{KC}\sum_{k=1}^{K}\left\|T_k(\mathbf{x}_i) - \widehat{F}_{g^\star}(\mathbf{x}_i)\right\|^2$, the mean squared deviation of individual-tree predictions from the forest prediction ($C = 1$ for regression), and the leaf support $u_i = \min_{1\le k\le K} s_k(L_k(\mathbf{x}_i))$, where $s_k(L_k(\mathbf{x}_i))$ is the training-sample count of the leaf of T_k reached by $\mathbf{x}_i$; it flags samples routed through thin, memorized regions of the trees. Each sample is assigned to a bucket $b_i = \big(\phi_d(d_i), \phi_u(u_i)\big)$ of an $m \times m$ grid, where ϕ_d and ϕ_u map d_i and u_i to quantile bins; the bin edges are estimated from the fitting subset defined below only, so calibration labels never influence the bucketing.

Correction structure. Each bucket b receives a policy $\pi_b = (h_b, \tau_b, \mathcal{A}_b)$, where $h_b \in \{\textsc{constant}, \textsc{linear}, \textsc{tree}\}$ is the corrector family, $\tau_b \in \mathcal{T} = \{0.5, 0.7, 0.9, 1\}$ is a within-bucket disagreement-rank threshold, and $\mathcal{A}_b$ is a feature subset. Features are ranked per bucket by the fraction of trees that split on them along the decision paths of the bucket's samples, and $\mathcal{A}_b$ is restricted to the top-a_b prefix of this ranking, with $1 \le a_b \le p_{\max}$ for the linear and tree families and $a_b = 0$ for the constant family. Every bucket therefore chooses from the same catalog Π of $|\mathcal{T}|(1 + 2p_{\max})$ policies. Only this discrete structure is selected; every numerical parameter is fitted from OOB residuals.

Cross-validated structure selection. The policy of each bucket is chosen by cross-validation on its own OOB residuals. For every candidate $\pi \in \Pi$, the rows of $\mathcal{I}_b$ (defined below) are split into three folds; a corrector with structure π is fitted on two folds with the trust weights and shrinkage of the next paragraph and predicts the residuals of the held-out fold. The bucket keeps the policy with the lowest cross-validated residual MSE,

$$\pi_b = \arg\min_{\pi\in\Pi} \frac{1}{n_b C}\sum_{f=1}^{3}\sum_{i\in\mathcal{I}_b^{(f)}} \left\|\mathbf{r}_i - \widehat{\boldsymbol{\delta}}_{\pi}^{(-f)}(\mathbf{x}_i)\right\|^2, \quad (7)$$

where $\mathcal{I}_b^{(f)}$ is fold f and $\widehat{\boldsymbol{\delta}}_{\pi}^{(-f)}$ is fitted on the other two, with ties broken toward simpler families, fewer features, and larger τ. Buckets with fewer than four rows keep a fixed default policy.

Data-fitted residual correction. We split $\mathcal{D}_{\mathrm{cor}}$ into a fitting subset $\mathcal{D}_{\mathrm{fit}}$ and a disjoint calibration subset $\mathcal{D}_{\mathrm{cal}}$ (25%, stratified for classification). Both are training rows of the forest: $\mathcal{D}_{\mathrm{cal}}$ is held out from bucketing, structure selection, and corrector fitting, but not from the bootstrap samples, so it is evaluated only through OOB predictions. Let $\mathcal{I}_b = \{i \in \mathcal{D}_{\mathrm{fit}} : b_i = b\}$ and $n_b = |\mathcal{I}_b|$. The within-bucket disagreement rank and trust weight are

$$q_i^{(b)} = \frac{1}{n_b}\sum_{j\in\mathcal{I}_b} \mathbf{1}[d_j \le d_i], \qquad \alpha_i = \begin{cases} 1, & q_i^{(b)} \le \tau_b, \\ 0.3, & q_i^{(b)} > \tau_b. \end{cases} \quad (8)$$

so that samples whose disagreement exceeds the bucket's trusted rank contribute only with weight 0.3. Let $\mathbf{W}_b = \mathrm{diag}(\alpha_i)$, $\mathbf{v}_i = [1, \mathbf{x}_{i,\mathcal{A}_b}^\top]^\top$, and let $\mathbf{V}_b$ and $\mathbf{R}_b$ stack $\mathbf{v}_i^\top$ and $\mathbf{r}_i^\top$, respectively. The bucket-specific corrector is

$$\widehat{\boldsymbol{\delta}}_b(\mathbf{x}) = \begin{cases} \dfrac{n_b \bar{\mathbf{r}}_b^{\alpha} + \lambda_{\mathrm{sh}}\bar{\mathbf{r}}}{n_b + \lambda_{\mathrm{sh}}}, & h_b = \textsc{constant}, \\ \mathbf{v}(\mathbf{x})^\top \widehat{\mathbf{B}}_b, & h_b = \textsc{linear}, \\ T_b^{\mathrm{res}}(\mathbf{x}_{\mathcal{A}_b}), & h_b = \textsc{tree}, \end{cases} \quad (9)$$

where

$$\begin{aligned} \bar{\mathbf{r}}_b^{\alpha} &= \frac{\sum_{i\in\mathcal{I}_b}\alpha_i \mathbf{r}_i}{\sum_{i\in\mathcal{I}_b}\alpha_i}, \\ \bar{\mathbf{r}} &= \frac{1}{|\mathcal{D}_{\mathrm{fit}}|}\sum_{i\in\mathcal{D}_{\mathrm{fit}}} \mathbf{r}_i, \\ \widehat{\mathbf{B}}_b &= (\mathbf{V}_b^\top \mathbf{W}_b \mathbf{V}_b)^{\dagger} \mathbf{V}_b^\top \mathbf{W}_b \mathbf{R}_b. \end{aligned} \quad (10)$$

The constant corrector shrinks the trust-weighted bucket mean toward the global mean residual with strength λ_{sh}, which stabilizes small buckets; the linear corrector is a trust-weighted least-squares fit with an intercept, using the pseudo-inverse for rank-deficient designs; the tree corrector T_b^{res} is a shallow regression tree (depth 3, minimum leaf size 5) trained on $\mathcal{A}_b$ with the same weights. The strength $\eta \in \{0, 0.05, 0.1, 0.2, 0.5, 1\}$ is then chosen by the MSE (regression) or log-loss (classification) on $\mathcal{D}_{\mathrm{cal}}$ of the OOB-based corrected prediction $\widehat{F}^{\mathrm{OOB}}_{g^\star}(\mathbf{x}_i) + \eta\widehat{\boldsymbol{\delta}}_{b_i}(\mathbf{x}_i)$, applied in logit space for classification as below; $\eta = 0$ lets calibration switch the correction off. With $b(\mathbf{x})$ the bucket of $\mathbf{x}$ and $\widehat{\boldsymbol{\delta}}_b = \mathbf{0}$ for buckets without fitting rows, the regression predictor is $\widehat{F}_{\mathrm{EvoGraft}}(\mathbf{x}) = \widehat{F}_{g^\star}(\mathbf{x}) + \eta\widehat{\boldsymbol{\delta}}_{b(\mathbf{x})}(\mathbf{x})$. For classification, the correction is instead applied in logit space: $\widehat{\mathbf{p}}_{\mathrm{EvoGraft}}(\mathbf{x}) = \mathrm{softmax}\Big(\mathbf{z}_{g^\star}(\mathbf{x}) + \eta\widehat{\boldsymbol{\delta}}_{b(\mathbf{x})}(\mathbf{x})\Big)$, where $\mathbf{z}_{g^\star} = \log\widehat{\mathbf{p}}_{g^\star} - \frac{1}{C}\sum_c \log\widehat{p}_{g^\star,c}$ is the centered-logit representation of the evolved forest probabilities; correcting in logit space rather than probability space keeps the corrected vector on the simplex after the final softmax, and $\widehat{\boldsymbol{\delta}}_b$ then has one component per class.

4 Experimental Results

4.1 Experimental Setting

Datasets and metrics. We evaluate EvoGraft on 14 datasets drawn from the CARTE benchmark [22], comprising seven classification tasks and seven regression tasks (Tables 1 and 2), which mix numerical and high-cardinality categorical attributes. We report macro-F1 for classification and root mean squared error (RMSE) for regression, computed on the transformed target scale when preprocessing transforms the target.

Data split. For each dataset, we first reserve 20% of the data as an *evolution pool*, used exclusively to search for the dataset-specific split-scoring program. This pool is divided into 75% development-training data, on which the candidate models are fitted, and 25% development-validation data, on which their evolutionary fitness is measured. The remaining 80% forms the *benchmark pool* shared by EvoGraft and all baselines. For each dataset, we use a fixed 75/25 train/test split of this pool, shared by all methods. We repeat training ten times while keeping this split fixed, varying the randomness used during training. The held-out benchmark test data are never used during split-score evolution, model fitting, residual-corrector selection, or calibration.

Compared baselines. We compare EvoGraft with baselines from three groups: (i) LLM-based methods, including DeLTa [37], MAPLE [21], and, for classification only, LLM-Tree [23] and FeatLLM [16]; (ii) tree-based methods, including Extra Trees (ET) [13], Random Forest (RF) [4], XGBoost (XGB) [8], and LightGBM (LGBM) [20]; and (iii) the tabular foundation model TabPFN-v2 [19].

4.2 Main Results

Tables 1 and 2 report macro-F1 and RMSE on the fourteen CARTE datasets. For each dataset we rank all methods with results by their mean score and report the mean rank across datasets.

Classification. EvoGraft has the lowest mean rank and the highest mean macro-F1 on Chocolate and Coffee. Its advantage is clearest on Coffee, where it exceeds the strongest baseline, LightGBM, by 5.4 percentage points. It remains below MAPLE on Michelin and Yelp, DeLTa on US Accidents, and both boosted ensembles on Whisky. On Spotify, its mean score is close to those of DeLTa and TabPFN-v2. These results support competitive classification performance across the tested tables, with specific weaknesses that the aggregate rank does not capture. FeatLLM and LLM-Tree show greater variation across training runs on most tables. These deviations reflect variability in training on a fixed data partition; the split-scoring programs remain frozen.

Regression. EvoGraft has the lowest mean rank and lower mean RMSE than DeLTa and the four tree ensembles on all seven regression datasets under the tested configurations. It leads on Cardekho and Museums. TabPFN-v2 has lower mean RMSE on four datasets, while MAPLE performs better on Anime Planet, Used Cars, and Wine Prices. EvoGraft therefore improves on the evaluated conventional tree baselines without replacing the strongest alternatives across all regression tasks.

4.3 Evolution Dynamics

Figure 2 traces the split-program search on the CARTE datasets. For every dataset and generation, it reports the fitness of the best program found so far relative to the best hand-designed criterion in the initial population, which contains CART, ID3, and C4.5-style criteria and balanced or hybrid variants of them. Fitness is measured on the development-validation split of the evolution pool. Relative to the best hand-designed initial criterion, the selected programs increase macro-F1 by 6.4% on average across classification datasets and reduce RMSE by 4.4% across regression datasets on this split. Over half of the average gain is reached within the first five generations, while later generations add smaller gains. The gain varies from under 1% on Used Cars to about 12% on Yelp, indicating that the search is more useful on some development splits than others. Because selection repeatedly uses these data and retains the best program found so far, the rising curves show progress on the search objective rather than independent evidence of better generalization.

4.4 Effect of the Evolved Split Criterion

To isolate the split-scoring program from the forest and the residual stage, we grow a single decision tree with the same tree builder and replace only its split criterion. We use seven numeric multi-class datasets with 6 to 26 classes, all obtained from the OpenML repository [35]: JapaneseVowels, Letter, Optdigits, Pendigits, Satimage, and Segment, which originate from the UCI repository [12], and Texture. We repeat training ten times, using either the full training set or $k \in \{128, 64, 32, 16\}$ labeled examples per class sampled from it. We compare the evolved program with four hand-designed criteria [27]: Gini impurity [5], entropy (information gain) [30], gain ratio [31], and class-weighted entropy, which weights each class by its inverse frequency at the parent node, analogous to the altered class priors of [5], and report each criterion's mean accuracy (Figure 3). The evolved program achieves higher mean accuracy than all evaluated human-designed criteria across the tested budgets, including the few-shot settings.

We hypothesize that this few-shot performance stems from the structural information available to the LLM through symbolic experience. Reports of representative trees, root-to-leaf feature interactions, weak leaves, and validation errors allow the LLM to relate split decisions to the behavior of the induced trees. When labeled examples are scarce, local impurity estimates can be noisy; this feedback may encourage scoring programs that balance purity with partition support and tree context, providing a useful inductive bias for learning from few examples.

We repeat the comparison with random forests of 100 trees of depth 6 on the same training subsets, with $k \in \{16, 32, 64, 128\}$ labeled examples per class and on the full training split (Figure 4); both forests share the tree builder and differ only in the split criterion. Replacing Gini impurity with the evolved program raises mean accuracy at every budget, by 0.8–1.6 percentage points, with gains on all seven datasets at 64 and 128 examples per class. The gains are smaller than for a single tree, as bagging already reduces the variance of individual splits.

4.5 Transfer of Split Programs Across Domains

We next ask whether a split program evolved on one dataset remains useful on datasets from a different domain. We take the seven programs evolved on the CARTE classification datasets and the seven evolved on the CARTE regression datasets and deploy each of them, without further search, on seven OpenML classification datasets (Vehicle, Steel Plates, Mfeat-Morphological, CMC, Eucalyptus, Vowel, and Credit-g) and seven OpenML regression datasets (Airfoil, Concrete, QSAR Fish, Space GA, White Wine, Geo Music, and House 16H, in the versions released with the DeLTa benchmark [37]). Unlike the CARTE tables, these datasets are mostly

Table 1: Macro-F1 (mean ± SD over ten training runs on a fixed train/test split; FeatLLM completed eight runs on Coffee and nine on Michelin/Spotify). Bold/underlined: best/second-best; lower rank is better.

Dataset	EvoGraft	LLM-Tree	DeLTa	FeatLLM	MAPLE	RF	ET	XGB	LGBM	TabPFN-v2
Chocolate	**0.597±0.011**	0.403±0.094	0.574±0.024	0.448±0.078	0.589±0.018	0.371±0.010	0.374±0.005	0.533±0.007	0.532±0.005	0.561±0.032
Coffee	**0.580±0.037**	0.448±0.075	0.502±0.065	0.487±0.099	0.383±0.008	0.376±0.003	0.379±0.007	0.525±0.012	0.526±0.004	0.378±0.005
Michelin	0.698±0.023	0.473±0.088	0.586±0.155	0.568±0.062	**0.715±0.009**	0.662±0.026	0.682±0.016	0.673±0.004	0.674±0.005	0.704±0.017
Spotify	0.776±0.001	0.479±0.119	**0.777±0.004**	0.726±0.006	0.770±0.003	0.719±0.032	0.725±0.016	0.768±0.001	0.769±0.001	0.776±0.007
US Accidents	0.698±0.006	0.429±0.075	**0.710±0.012**	0.609±0.050	0.706±0.004	0.620±0.024	0.620±0.025	0.698±0.001	0.699±0.002	0.625±0.027
Whisky	0.755±0.010	0.518±0.107	0.712±0.028	0.614±0.167	0.725±0.020	0.723±0.023	0.707±0.020	0.774±0.008	**0.779±0.004**	0.758±0.023
Yelp	0.650±0.001	0.443±0.072	0.642±0.050	0.611±0.015	**0.662±0.006**	0.349±0.001	0.349±0.001	0.643±0.002	0.644±0.002	0.638±0.011
Mean rank ↓	**2.43**	8.86	4.14	7.57	3.14	8.43	7.71	4.43	3.57	4.71

Table 2: RMSE (mean ± SD over ten training runs on a fixed train/test split). Bold/underlined: best/second-best; lower RMSE and rank are better.

Dataset	EvoGraft	DeLTa	MAPLE	RF	ET	XGB	LGBM	TabPFN-v2
Anime Planet	0.669±0.011	0.731±0.028	0.664±0.006	0.839±0.005	0.850±0.005	0.691±0.001	0.692±0.001	**0.656±0.019**
Beer Ratings	0.317±0.003	0.475±0.032	0.332±0.004	0.391±0.003	0.401±0.002	0.347±0.000	0.345±0.001	**0.302±0.005**
Cardekho	**0.073±0.000**	0.075±0.001	0.078±0.000	0.145±0.004	0.149±0.003	0.104±0.000	0.104±0.000	0.095±0.010
Employee Sal.	0.099±0.003	0.120±0.007	0.101±0.002	0.152±0.003	0.153±0.003	0.122±0.000	0.122±0.000	**0.084±0.005**
Museums	**0.479±0.007**	0.530±0.050	0.504±0.003	0.614±0.004	0.615±0.004	0.504±0.001	0.504±0.001	0.496±0.027
Used Cars	0.112±0.003	0.121±0.006	**0.106±0.001**	0.158±0.002	0.166±0.002	0.126±0.000	0.126±0.000	0.130±0.005
Wine Prices	0.728±0.010	0.780±0.028	0.724±0.009	1.032±0.007	1.043±0.008	0.842±0.000	0.850±0.000	**0.667±0.016**
Mean rank ↓	**2.00**	4.71	2.71	6.86	7.86	4.86	4.71	2.29

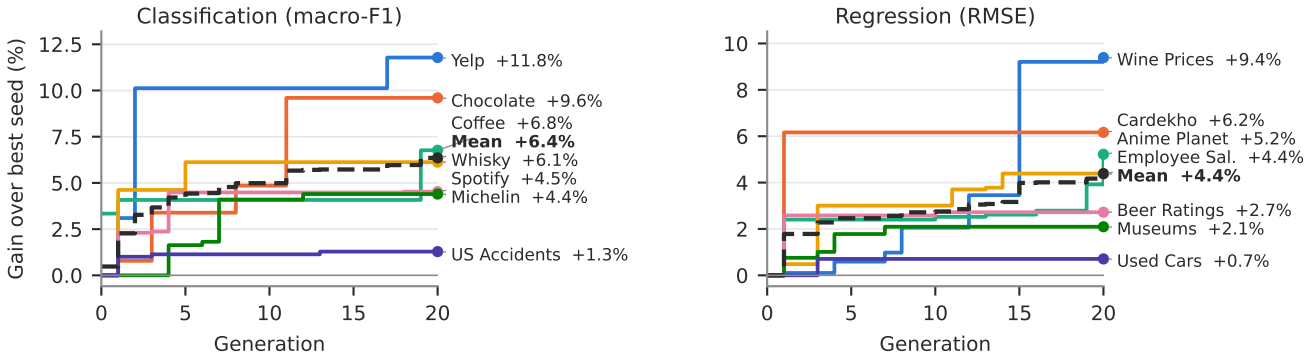


Figure 2: Split-program evolution on the seven classification (left) and seven regression (right) CARTE datasets: improvement of the best program found so far over the best hand-designed criterion in the initial population, on the development-validation split of the evolution pool. Each coloured line is one dataset, labelled at its right end with its final gain; the dashed line is the mean over datasets.

numeric, and they differ in size and in the number of classes. Figure 5 reports the differences averaged over ten training runs on each target dataset.

Transfer is uneven across datasets. In classification, all transferred programs improve performance on Steel Plates but reduce it on CMC, while their effects vary on Mfeat-Morphological and Eucalyptus. In regression, transfer provides no aggregate improvement: the programs reduce error on average on Airfoil but increase it on Concrete, Space GA, and House 16H. The outcome also depends on the source–target pair: on Concrete, the program evolved on Museums reduces error, while the Wine Prices program increases it. These results highlight the importance of evolving split-scoring programs separately for each dataset, so that split decisions reflect its statistical and structural characteristics.

4.6 Split Programs with Engineered Features

The split programs are evolved on the original attributes of each dataset. We therefore ask how EvoGraft performs when its forest is grown on a richer feature set produced by feature engineering. We consider two LLM-based generators, LLM-FE [1], which evolves

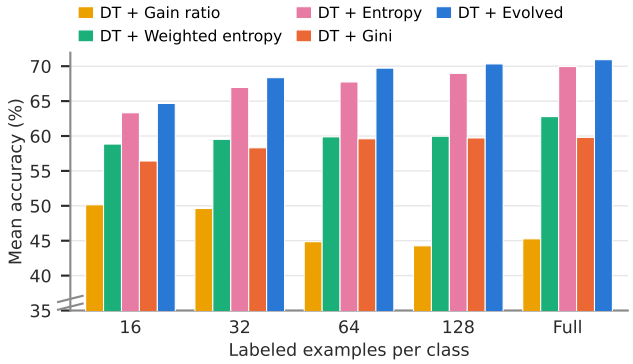


Figure 3: Single-tree accuracy on seven OpenML datasets, averaged over ten training runs, for k labels per class and full training data, with a decision tree (DT) grown with each split criterion.

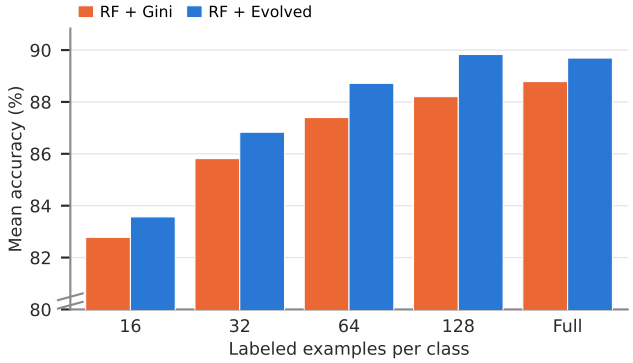


Figure 4: Random-forest (RF) accuracy on seven OpenML datasets, averaged over ten training runs, comparing Gini and evolved scores with k labels per class or full training data.

a feature-transformation program, and OCTree [28], which proposes rule-based features, and OpenFE [40], an automated feature generator that does not use an LLM. Each generator is run once per dataset on the fixed training split, and its features are frozen and reused in all ten training runs. EvoGraft is then fitted on the resulting features with the same split program and residual stage as in Tables 1 and 2, without further evolution (Table 3). LLM-FE and OpenFE generally improve EvoGraft, whereas OCTree provides no consistent benefit, suggesting that gains depend on feature quality. The evolved split programs can exploit the richer inputs without further evolution. These results suggest that feature engineering and split-program evolution are complementary: one enriches the inputs, while the other guides their partitioning, allowing EvoGraft to benefit from both.

Table 3: Feature engineering with EvoGraft on CARTE (mean ± SD over ten training runs on a fixed train/test split). Bold/underlined: best/second-best.

Dataset	EvoGraft	+ LLM-FE	+ OCTree	+ OpenFE
Classification: macro-F1 (higher is better)				
Chocolate	0.597±0.011	**0.680±0.008**	0.577±0.011	0.654±0.014
Coffee	0.580±0.037	**0.752±0.009**	0.510±0.047	0.624±0.022
Michelin	0.698±0.023	**0.780±0.008**	0.714±0.018	0.734±0.007
Spotify	0.776±0.001	0.773±0.002	0.777±0.002	**0.851±0.002**
US Accidents	0.698±0.006	**0.715±0.004**	0.696±0.006	0.712±0.004
Whisky	0.755±0.010	**0.773±0.009**	0.757±0.009	0.765±0.012
Yelp	0.650±0.001	**0.710±0.001**	0.646±0.002	0.697±0.002
Regression: RMSE (lower is better)				
Anime Planet	0.669±0.011	**0.613±0.006**	0.665±0.013	0.629±0.002
Beer Ratings	**0.317±0.003**	0.329±0.002	0.317±0.004	0.318±0.003
Cardekho	0.073±0.000	**0.067±0.002**	0.072±0.000	0.071±0.000
Employee Sal.	0.099±0.003	0.096±0.002	0.102±0.002	**0.095±0.002**
Museums	0.479±0.007	**0.412±0.002**	0.492±0.021	0.438±0.009
Used Cars	0.112±0.003	**0.084±0.001**	0.113±0.004	0.105±0.002
Wine Prices	0.728±0.010	**0.693±0.008**	0.721±0.005	0.698±0.007

4.7 Ablation Study

Table 4 changes one design choice of EvoGraft at a time on the seven OpenML classification datasets of Section 4.4 and the seven CARTE regression datasets. Each variant is evaluated over ten training runs, with matching training and test data across variants.

Residual corrector. We remove the residual stage and evaluate the evolved forest on its own. Performance deteriorates in both classification and regression, indicating that systematic prediction errors remain after split-score evolution. OOB residual correction therefore complements the evolved tree construction by addressing errors in the forest's aggregated predictions.

Evolved split score. We keep the corrector and replace the evolved program with Gini impurity for classification and variance reduction for regression. In classification, the evolved score yields higher mean accuracy, and in regression it yields lower mean RMSE, even when residual correction is retained. This shows that adapting the split objective contributes beyond post-hoc correction, consistent with Section 4.4, and supports adapting both tree construction and forest-level prediction.

Structure selection. We replace the cross-validated choice of each bucket's corrector structure (Eq. 7) by a random choice from Π or by a heuristic that uses all top-$p_{\max}$ features, $\tau_b = 1$, and a constant or linear corrector depending only on the bucket size. Both alternatives yield lower mean accuracy and higher mean RMSE than cross-validation, with a clearer benefit from cross-validation in regression. Its frequent selection of tree correctors, which the heuristic excludes, suggests that flexibility in the correction family is useful. These results support selecting correction structures from OOB evidence to match the residual patterns of each bucket.

Bucket grid size. We vary the grid size m, which sets how many uncertainty buckets receive their own corrector; $m = 1$ fits a single corrector for the whole forest and EvoGraft uses $m = 3$. Multiple buckets outperform a single global corrector in both tasks. These

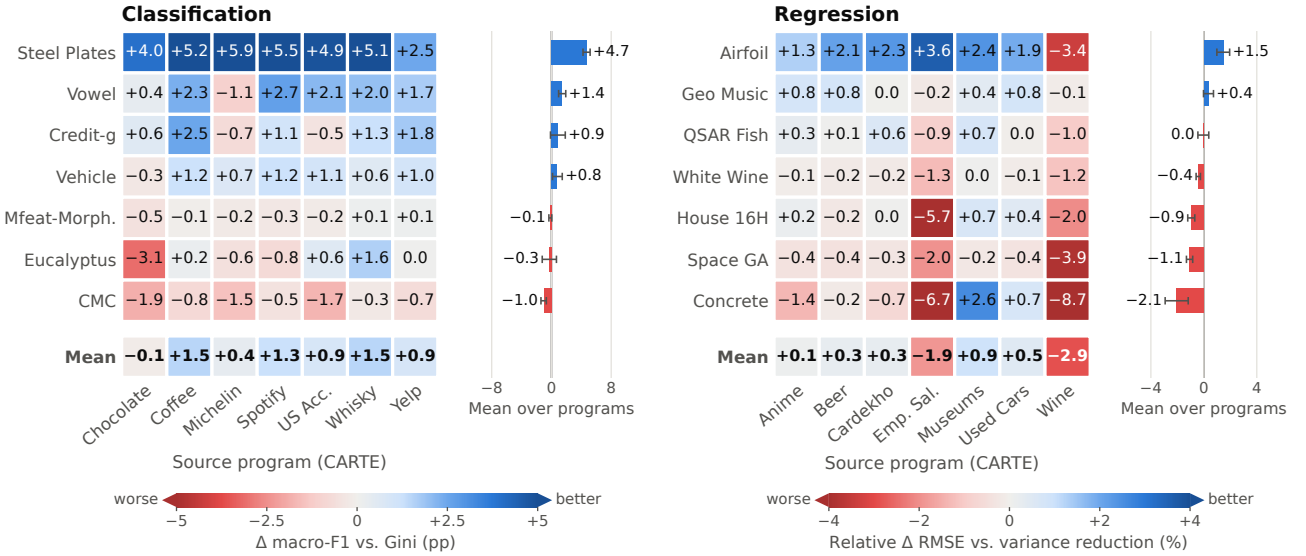


Figure 5: CARTE-to-OpenML transfer gains over Gini/variance reduction, averaged over ten training runs. Columns/rows: sources/targets; side bars: target means ± SE; bottom rows: source means.

results support adapting correction to different uncertainty regions, with the appropriate granularity depending on the task.

Table 4: Ablation: mean accuracy/RMSE on OpenML/CARTE over ten training runs. Gini: fixed Gini/variance reduction. Bold/underlined: best/second-best.

Ablation	Variant	Accuracy ↑	RMSE ↓
Corrector	Removed	0.832	0.374
Split score	Gini	0.836	0.356
Structure	Random	0.840	0.367
	Heuristic	0.847	0.364
Grid size	$m = 1$	0.837	0.357
	$m = 2$	0.845	0.356
EvoGraft (evolved, CV, $m = 3$)		**0.850**	**0.354**

5 Conclusion

We presented EvoGraft, a framework that adapts tree construction through dataset-specific split-score evolution and forest predictions through uncertainty-aware OOB residual correction. Symbolic experience connects LLM-generated scoring programs to the structures and errors of the induced trees, while OOB evidence guides correction after forest fitting. On fourteen tabular benchmarks, EvoGraft obtains the lowest mean rank among the evaluated methods in both classification and regression under the tested configurations. Its advantages remain dataset-dependent, and uneven transfer highlights the importance of evolving split scores for individual datasets.

Several limitations remain. First, split-score evolution requires repeated model fitting and LLM calls, adding a search cost whose usefulness depends on the available budget and the quality of generated programs. Second, repeated selection on development-validation data may favor programs tailored to those data, and a program evolved on one dataset may not transfer reliably to another. Finally, correctors are restricted to a fixed catalog of constant, linear, and shallow tree models, discrete trust thresholds, and ranked feature subsets.

Future work could improve search efficiency, evaluate programs across more varied development splits, and adapt bucket granularity to the available OOB evidence. Expanding or evolving the set of correction models is another direction, while preserving the separation between structure selection, parameter fitting, and calibration.

References

[1] Nikhil Abhyankar, Parshin Shojaee, and Chandan K. Reddy. 2026. LLM-FE: Automated Feature Engineering for Tabular Data with LLMs as Evolutionary Optimizers. arXiv:2503.14434 [cs.LG] https://arxiv.org/abs/2503.14434

[2] Rodrigo C. Barros, Márcio P. Basgalupp, André C. P. L. F. de Carvalho, and Alex A. Freitas. 2012. A Hyper-Heuristic Evolutionary Algorithm for Automatically Designing Decision-Tree Algorithms. In *Proceedings of the 14th Annual Conference on Genetic and Evolutionary Computation*. ACM, 1237–1244. doi:10.1145/2330163.2330335

[3] Márcio Porto Basgalupp, Rodrigo Coelho Barros, and Tiago Barabasz. 2014. A Grammatical Evolution Based Hyper-Heuristic for the Automatic Design of Split Criteria. In *Proceedings of the 2014 Annual Conference on Genetic and Evolutionary Computation*. ACM, 1311–1318. doi:10.1145/2576768.2598327

[4] Leo Breiman. 2001. Random Forests. *Machine Learning* 45, 1 (2001), 5–32. doi:10.1023/A:1010933404324

[5] Leo Breiman, Jerome H. Friedman, Richard A. Olshen, and Charles J. Stone. 1984. *Classification and Regression Trees*. Wadsworth & Brooks/Cole.

[6] Thyago P. Carvalho, Fabrízzio A. A. M. N. Soares, Roberto Vita, Roberto da Piedade Francisco, João Pedro Tavares Vieira Basto, and Symone Gomes Soares Alcalá. 2019. A Systematic Literature Review of Machine Learning Methods Applied to Predictive Maintenance. *Computers & Industrial Engineering* 137 (2019), 106024. doi:10.1016/j.cie.2019.106024

[7] Chentong Chen, Mengyuan Zhong, Ye Fan, Jialong Shi, and Jianyong Sun. 2026. HiFo-Prompt: Prompting with Hindsight and Foresight for LLM-based Automatic Heuristic Design. In *International Conference on Learning Representations*. https://openreview.net/forum?id=imSLzfZ6av

[8] Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 785–794. doi:10.1145/2939672.2939785

[9] Andrea Dal Pozzolo, Giacomo Boracchi, Olivier Caelen, Cesare Alippi, and Gianluca Bontempi. 2017. Credit Card Fraud Detection: A Realistic Modeling and a Novel Learning Strategy. *IEEE Transactions on Neural Networks and Learning Systems* 29, 8 (2017), 3784–3797. doi:10.1109/TNNLS.2017.2736643

[10] Pham Vu Tuan Dat, Long Doan, and Huynh Thi Thanh Binh. 2025. HSEvo: Elevating Automatic Heuristic Design with Diversity-Driven Harmony Search and Genetic Algorithm Using LLMs. *Proceedings of the AAAI Conference on Artificial Intelligence* 39, 25 (2025), 26931–26938. doi:10.1609/aaai.v39i25.34898

[11] Tuan Dinh, Yuchen Zeng, Ruisu Zhang, Ziqian Lin, Michael Gira, Shashank Rajput, Jy-yong Sohn, Dimitris Papailiopoulos, and Kangwook Lee. 2022. LIFT: Language-Interfaced Fine-Tuning for Non-Language Machine Learning Tasks. In *Advances in Neural Information Processing Systems*, Vol. 35. 11763–11784. doi:10.52202/068431-0855

[12] Dheeru Dua and Casey Graff. 2019. UCI Machine Learning Repository. University of California, Irvine, School of Information and Computer Sciences. http://archive.ics.uci.edu/ml

[13] Pierre Geurts, Damien Ernst, and Louis Wehenkel. 2006. Extremely Randomized Trees. *Machine Learning* 63, 1 (2006), 3–42. doi:10.1007/s10994-006-6226-1

[14] Yury Gorishniy, Ivan Rubachev, Valentin Khrulkov, and Artem Babenko. 2021. Revisiting Deep Learning Models for Tabular Data. In *Advances in Neural Information Processing Systems*, Vol. 34. 18932–18943. https://proceedings.neurips.cc/paper/2021/hash/9d86d83f925f2149e9edb0ac3b49229c-Abstract.html

[15] Léo Grinsztajn, Edouard Oyallon, and Gaël Varoquaux. 2022. Why Do Tree-Based Models Still Outperform Deep Learning on Typical Tabular Data?. In *Advances in Neural Information Processing Systems*, Vol. 35. 507–520. doi:10.52202/068431-0037

[16] Sungwon Han, Jinsung Yoon, Sercan O. Arik, and Tomas Pfister. 2024. Large Language Models Can Automatically Engineer Features for Few-Shot Tabular Learning. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, 17454–17479. https://proceedings.mlr.press/v235/han24f.html

[17] Stefan Hegselmann, Alejandro Buendia, Hunter Lang, Monica Agrawal, Xiaoyi Jiang, and David Sontag. 2023. TabLLM: Few-Shot Classification of Tabular Data with Large Language Models. In *Proceedings of the 26th International Conference on Artificial Intelligence and Statistics (Proceedings of Machine Learning Research, Vol. 206)*. PMLR, 5549–5581. https://proceedings.mlr.press/v206/hegselmann23a.html

[18] Noah Hollmann, Samuel Müller, and Frank Hutter. 2023. Large Language Models for Automated Data Science: Introducing CAAFE for Context-Aware Automated Feature Engineering. In *Advances in Neural Information Processing Systems*, Vol. 36. doi:10.52202/075280-1938

[19] Noah Hollmann, Samuel Müller, Lennart Purucker, Arjun Krishnakumar, Max Körfer, Shi Bin Hoo, Robin Tibor Schirrmeister, and Frank Hutter. 2025. Accurate Predictions on Small Data with a Tabular Foundation Model. *Nature* 637, 8045 (2025), 319–326. doi:10.1038/s41586-024-08328-6

[20] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. 2017. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In *Advances in Neural Information Processing Systems*, Vol. 30. 3146–3154.

[21] Nguyen Viet Tuan Kiet, Nguyen Ba Thinh, Thanh Trung Huynh, and Hieu Pham. 2026. MAPLE: Multi-Agent Prior Learning for Constructing Tree Ensembles. In *Workshop on Multi-Agent Learning and Its Opportunities in the Era of Generative AI*. https://openreview.net/forum?id=bBIA4TVEBG

[22] Myung Jun Kim, Léo Grinsztajn, and Gaël Varoquaux. 2024. CARTE: Pretraining and Transfer for Tabular Learning. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, 23843–23866. https://proceedings.mlr.press/v235/kim24d.html

[23] Ricardo Knauer, Mario Koddenbrock, Raphael Wallsberger, Nicholas M. Brisson, Georg N. Duda, Deborah Falla, David W. Evans, and Erik Rodner. 2025. "Oh LLM, I'm Asking Thee, Please Give Me a Decision Tree": Zero-Shot Decision Tree Induction and Embedding with Large Language Models. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '25)*. ACM, 1196–1206. doi:10.1145/3711896.3736818

[24] Stefan Lessmann, Bart Baesens, Hsin-Vonn Seow, and Lyn C. Thomas. 2015. Benchmarking State-of-the-Art Classification Algorithms for Credit Scoring: An Update of Research. *European Journal of Operational Research* 247, 1 (2015), 124–136. doi:10.1016/j.ejor.2015.05.030

[25] Fei Liu, Xialiang Tong, Mingxuan Yuan, Xi Lin, Fu Luo, Zhenkun Wang, Zhichao Lu, and Qingfu Zhang. 2024. Evolution of Heuristics: Towards Efficient Automatic Algorithm Design Using Large Language Model. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, 32201–32223. https://proceedings.mlr.press/v235/liu24bs.html

[26] Tennison Liu, Nicolas Huynh, and Mihaela van der Schaar. 2025. Decision Tree Induction Through LLMs via Semantically-Aware Evolution. In *The Thirteenth International Conference on Learning Representations*. https://openreview.net/forum?id=UyhRtB4hjN

[27] John Mingers. 1989. An Empirical Comparison of Selection Measures for Decision-Tree Induction. *Machine Learning* 3, 4 (1989), 319–342.

[28] Jaehyun Nam, Kyuyoung Kim, Seunghyuk Oh, Jihoon Tack, Jaehyung Kim, and Jinwoo Shin. 2024. Optimized Feature Generation for Tabular Data via LLMs with Decision Tree Reasoning. In *Advances in Neural Information Processing Systems*, Vol. 37. doi:10.52202/079017-2932

[29] Jaehyun Nam, Woomin Song, Seong Hyeon Park, Jihoon Tack, Sukmin Yun, Jaehyung Kim, Kyu Hwan Oh, and Jinwoo Shin. 2024. Tabular Transfer Learning via Prompting LLMs. In *Conference on Language Modeling*. https://openreview.net/forum?id=bo4pauxnIR

[30] J. Ross Quinlan. 1986. Induction of Decision Trees. *Machine Learning* 1, 1 (1986), 81–106.

[31] J. Ross Quinlan. 1993. *C4.5: Programs for Machine Learning*. Morgan Kaufmann.

[32] Alvin Rajkomar, Eyal Oren, Kai Chen, Andrew M. Dai, Nissan Hajaj, Michaela Hardt, Peter J. Liu, Xiaobing Liu, Jake Marcus, Mimi Sun, Patrik Sundberg, Hector Yee, Kun Zhang, Yi Zhang, Gerardo Flores, Gavin E. Duggan, Jamie Irvine, Quoc Le, Kurt Litsch, Alexander Mossin, Justin Tansuwan, De Wang, James Wexler, Jimbo Wilson, Dana Ludwig, Samuel L. Volchenboum, Katherine Chou, Michael Pearson, Srinivasan Madabushi, Nigam H. Shah, Atul J. Butte, Michael D. Howell, Claire Cui, Greg S. Corrado, and Jeffrey Dean. 2018. Scalable and Accurate Deep Learning with Electronic Health Records. *npj Digital Medicine* 1, 1 (2018), 18. doi:10.1038/s41746-018-0029-1

[33] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. 2024. Mathematical Discoveries from Program Search with Large Language Models. *Nature* 625, 7995 (2024), 468–475. doi:10.1038/s41586-023-06924-6

[34] Dylan Slack and Sameer Singh. 2023. TABLET: Learning From Instructions For Tabular Data. *CoRR* abs/2304.13188 (2023). doi:10.48550/arXiv.2304.13188

[35] Joaquin Vanschoren, Jan N. van Rijn, Bernd Bischl, and Luis Torgo. 2013. OpenML: Networked Science in Machine Learning. *SIGKDD Explorations* 15 (2013), 49–60. doi:10.1145/2641190.2641198

[36] Zhihan Yang, Jiaqi Wei, Xiang Zhang, Haoyu Dong, Yiwen Wang, Xiaoke Guo, Pengkun Zhang, Yiwei Xu, and Chenyu You. 2026. FORESTLLM: Large Language Models Make Random Forest Great on Few-shot Tabular Learning. arXiv:2601.11311 [cs.LG] doi:10.48550/arXiv.2601.11311

[37] Hangting Ye, Jinmeng Li, He Zhao, Dandan Guo, and Yi Chang. 2025. LLM Meeting Decision Trees on Tabular Data. In *Advances in Neural Information Processing Systems*, Vol. 38. doi:10.52202/085713-3938

[38] Haoran Ye, Jiarui Wang, Zhiguang Cao, Federico Berto, Chuanbo Hua, Haeyeon Kim, Jinkyoo Park, and Guojie Song. 2024. ReEvo: Large Language Models as Hyper-Heuristics with Reflective Evolution. In *Advances in Neural Information Processing Systems*, Vol. 37. 43571–43608. doi:10.52202/079017-1381

[39] Yifei Yuan, Jiatong Li, Weijia Zhang, Mohammad Aliannejadi, Evangelos Kanoulas, and Renjun Hu. 2025. Summarize-Exemplify-Reflect: Data-driven Insight Distillation Empowers LLMs for Few-shot Tabular Classification. In *Findings of the Association for Computational Linguistics: EMNLP 2025*. Association for Computational Linguistics, 12324–12348. doi:10.18653/v1/2025.findings-emnlp.659

[40] Tianping Zhang, Zheyu Zhang, Zhiyuan Fan, Haoyan Luo, Fengyuan Liu, Qian Liu, Wei Cao, and Jian Li. 2023. OpenFE: Automated Feature Generation with Expert-level Performance. arXiv:2211.12507 [cs.LG] https://arxiv.org/abs/2211.12507

[41] Zhi Zheng, Zhuoliang Xie, Zhenkun Wang, and Bryan Hooi. 2025. Monte Carlo Tree Search for Comprehensive Exploration in LLM-Based Automatic Heuristic Design. In *Proceedings of the 42nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 267)*. PMLR, 78338–78373. https://proceedings.mlr.press/v267/zheng25o.html

Appendix

Evolving Dataset-Specific Split Scores and Out-of-Bag Residual Correction for Tree Ensembles

Table of Contents

A Prompt Details **2**
- A.1 Search Configuration . . . 2
- A.2 System Prompt . . . 2
- A.3 User Prompt . . . 4
- A.4 Operator Instructions . . . 4
- A.5 Symbolic Experience . . . 5
- A.6 Repair Prompt . . . 6

B Evolved Split Programs **7**
- B.1 Classification Programs . . . 7
- B.2 Regression Programs . . . 12

C Selected Correction Structures **16**

D Additional Results for the Main Experiments **17**
- D.1 Training and Inference Time . . . 17
- D.2 LLM Token Usage . . . 18
- D.3 Statistical Significance . . . 18

A Prompt Details

Every LLM call in the split-program search consists of one system message and one user message. The system message is fixed for a task type; the user message names the variation operator, gives its instruction, and lists the parent programs with their fitness and symbolic experience. All listings below are rendered from the code that ran the CARTE experiments, with the settings of those runs: node-context programs, symbolic reflection enabled, macro-F1 fitness for classification and RMSE fitness for regression. Braced upper-case terms such as `{PARENT_1_SOURCE}` are filled in at run time. The prompts refer to the method by its internal name, ESLARA Evolved. Line breaks inside the listings are typographical.

A.1 Search Configuration

Table 1 lists the settings shared by all fourteen evolution runs. Fitness is computed on the development-validation split of the evolution pool with a smaller bootstrap forest of the deployed depth.

Table 1: Split-program search configuration, shared by the fourteen CARTE runs.

Setting	Value
LLM	GLM-5.3, temperature 1.0
Population size P / elite pool	10 / 4
Offspring per generation	8
Generations / maximum evaluations	20 / 170
Parents for E1 and E2 / for M1–M3	3 / 1
Response retries (repair prompts)	3
Program limits	512 operations, AST depth 40
Fitness forest	10 trees, depth 6, all features, bootstrap
Fitness metric	macro-F1 (classification), RMSE (regression)
Symbolic experience	2 representative trees, 3 interaction pairs, 4 disagreement samples
Search seed	7

A.2 System Prompt

The classification system prompt is reproduced in full. It defines the task, the program interface and the node context, the restricted Python dialect in which programs are written, the search and data boundaries, and a syntax example.

System prompt (classification)

```
You design a node-adaptive split-scoring program for the classification forest in ESLARA Evolved.
This program controls CART-style tree splits inside the forest. Residual correction is a separate
component and is not part of the reported search fitness. Optimize only the declared development
validation metric. Evolution uses a reserved development pool. The separate benchmark train and
test partitions are unavailable and must never influence the program or its selection.
One complete Python function is used to score every candidate split, but it may change its behavior
according to the current node and candidate context. Model fitness is measured on validation data;
higher returned split scores are preferred inside CART.

Return only one plain Python source snippet. Its first line must be exactly one single-line comment
in the form `# Thought: <one-sentence rationale>`, followed by one function. Do not use Markdown
fences or prose outside the snippet. The signature must be exactly:
def score_split(y_left, y_right, context):

y_left and y_right are the non-empty target-label tuples induced by this candidate. Their concatenation
is the current node's target multiset. context is a read-only mapping with this exact schema:

Node position and scale:
- context["depth"]: int, root is 0.
- context["max_depth"]: positive int, configured CART depth limit.
- context["depth_fraction"]: float depth / max_depth, normally in [0, 1].
- context["node_sample_count"]: int, training occurrences reaching the node.
- context["root_sample_count"]: int, training occurrences at the current tree's root.
- context["node_fraction"]: float node_sample_count / root_sample_count, in (0, 1].

Feature and candidate identity:
- context["feature_index"]: zero-based int column index.
- context["feature_name"]: runtime string column name; do not hard-code a particular name.
- context["feature_kind"]: "numeric" or "categorical".
```

```
- context["split_kind"]: "numeric" or "categorical" for this candidate.
- context["threshold"]: float for a numeric <= split, otherwise None.
- context["category"]: string for a categorical == split, otherwise None.
- context["candidate_index"]: zero-based ordinal among valid candidates for this feature.
- context["candidate_count"]: positive int number of valid candidates considered for this feature.

Path and construction history, measured before selecting the current split:
- context["feature_use_count"]: int prior selections of this feature while building the current tree.
- context["path_feature_use_count"]: int occurrences of this feature on the root-to-parent path.
- context["ancestor_feature_indices"]: tuple of root-to-parent feature indices, preserving repeats/order.
- context["total_internal_splits"]: int splits already selected in the current tree's depth-first order.

Candidate geometry and task metadata:
- context["left_fraction"] and context["right_fraction"] are child sizes divided by node size; both
  are in (0, 1) and sum to 1.
- context["balance"] is 2 * min(left_size, right_size) / node_size, in (0, 1].
- context["n_features"]: positive int total input columns.
- context["n_classes"]: positive int number of target classes for classification.
- context["task_type"]: the string "classification" in this prompt.

All context values come only from the current training node/candidate before it is selected. They
contain no validation/test labels, predictions, or fitness values. Raw feature vectors are not
provided; derive target statistics from y_left/y_right and use context only for node/candidate adaptation.
The schema is backend-invariant: it contains no backend name, ensemble round, tree index, bootstrap
identifier, or accumulated state from earlier trees. root_sample_count and both construction-history
counters reset independently for every tree, so a CART root and each RF/XGB tree root share the same
coordinate semantics even when their sampled rows differ.

Write the complete scoring algorithm, not merely an arithmetic expression. You may use local
variables, multiple returns, if statements, bounded for loops over supplied finite collections,
read-only indexing, slices, collection literals, comprehensions, generators, arithmetic, comparisons, and
boolean logic. Safe calls include len, sum, min, max, abs, all, any, set, sorted, enumerate, zip,
list, tuple, float, int, bool, str, round, log, log1p, sqrt, exp, sin, cos, tanh, floor, ceil,
clamp, mean, median, and variance. Implement any impurity or gain logic directly from the inputs.
Do not import modules, perform I/O or network access, mutate the inputs/context, use global state,
attribute access, while loops, recursion, exceptions, or unlisted calls. Do not hard-code dataset
feature names; reason from the runtime context. Return a finite float for every valid candidate.

Executable DSL contract (this is restricted Python, not unrestricted Python):
- Define exactly one score_split function. Inline all statistics in its body: no nested def,
  helper functions, lambda, imports, classes, decorators, annotations, or default arguments.
- Assign only to local variable names, including scalar += updates or local-name unpacking.
  Never assign to a subscript, even in a newly created local collection: counts[label] = value,
  counts[label] += 1, values[i] = value, and context["balance"] = value are invalid.
  Build counts without mutation, for example:
  labels = set(list(y_left) + list(y_right))
  left_counts = [sum(1 for value in y_left if value == label) for label in labels]
- Call only these bare names: abs, all, any, bool, ceil, clamp, cos, enumerate, exp, float, floor, int, len, list, log, log1p, max, mean, median, min, round, set,
       sin, sorted, sqrt, str, sum, tanh, tuple, variance, zip. Functions are already provided; do not import them.
  Use positional arguments only. No attribute or method calls: context.get, list.append,
  dict.items, list.count, math.log, and numpy functions are invalid.
- In node_context mode use context["depth"] or context["balance"] to read documented keys.
  All documented keys are present. Do not invent keys or use context.get. In target_only mode
  there is no context argument. Never reassign input arguments or safe function names.
- Iterate only over supplied finite collections and their derived collections. range is not
  available. At most two nested loop/comprehension levels are allowed; a comprehension may
  have at most two generators. Prefer short direct sums over elaborate data structures.
- No standalone expression statements, while, try, raise, with, yield, global, nonlocal,
  assignment expressions (:=), starred unpacking, I/O, network access, or input mutation.
- Guard zero denominators and degenerate pure nodes. Return a finite float on every path.
- Limits: at most 512 static operations and AST depth 40. Numeric constants must be finite.
Before returning code, check this contract and the exact signature silently.

Meaningful search and data boundaries:
The split score must change the ordering of competing candidates to change a tree. Multiplying a parent score by a positive factor shared by every candidate at a
       fixed node (such as a depth-only or node-size-only factor) leaves that ordering unchanged. A strictly increasing transform also preserves ordering. Instead
       change the relative contribution of candidate-specific target statistics, balance, or feature/path usage. Depth may modulate those relative weights. With
       few-shot training, prefer compact stable changes.
Symbolic experience, when supplied, comes only from development training and validation. Treat its auxiliary metrics as diagnostics; optimize the declared fitness
       metric. Do not encode validation rows, labels, feature names, thresholds, or identifiers into the source. The function must derive its score from the
       current training candidate inputs.

Syntax example (not a required heuristic; do not simply copy it):
def score_split(y_left, y_right, context):
    parent = list(y_left) + list(y_right)
    n_left = float(len(y_left))
```

```
    n_right = float(len(y_right))
    total = n_left + n_right
    labels = set(parent)
    parent_probs = [sum(1 for value in parent if value == label) / total for label in labels]
    left_probs = [sum(1 for value in y_left if value == label) / n_left for label in labels]
    right_probs = [sum(1 for value in y_right if value == label) / n_right for label in labels]
    parent_impurity = 1.0 - sum(probability ** 2 for probability in parent_probs)
    left_impurity = 1.0 - sum(probability ** 2 for probability in left_probs)
    right_impurity = 1.0 - sum(probability ** 2 for probability in right_probs)
    if parent_impurity <= 0.0:
        return 0.0
    weighted_impurity = (n_left * left_impurity + n_right * right_impurity) / total
    gain = (parent_impurity - weighted_impurity) / parent_impurity
    balance = context["balance"]
    return float(gain * (0.8 + 0.2 * balance))
```

The regression system prompt is identical except for the following lines, which name the regression forest, describe the targets as numeric, set the class count to zero, and use variance in place of Gini impurity in the syntax example.

System prompt (regression): lines that differ

```
You design a node-adaptive split-scoring program for the regression forest in ESLARA Evolved.
y_left and y_right are the non-empty numeric-target tuples induced by this candidate. Their concatenation
- context["n_classes"]: 0 for regression.
- context["task_type"]: the string "regression" in this prompt.
    parent_mean = sum(parent) / total
    left_mean = sum(y_left) / n_left
    right_mean = sum(y_right) / n_right
    parent_impurity = sum((value - parent_mean) ** 2 for value in parent) / total
    left_impurity = sum((value - left_mean) ** 2 for value in y_left) / n_left
    right_impurity = sum((value - right_mean) ** 2 for value in y_right) / n_right
```

A.3 User Prompt

The user message wraps the operator instruction and the parent blocks. The listing shows a complete M1 user prompt for classification; E1 and E2 list three parent blocks instead of one, and regression prompts state the objective as `RMSE, minimize`.

User prompt (operator M1, classification)

````
ESLARA Evolved evolutionary operator: M1
Validation objective: MACRO_F1, maximize.

Study the reported validation errors, weak leaves, node diagnostics, and symbolic tree paths. Refine the parent split score to address one supported failure
       pattern while preserving successful behavior. A report can contain just one CART tree; best and worst representatives can then be the same tree. Do not
       assume unreported ensemble diversity.

Existing parent heuristics:
Parent 1
Thought: {PARENT_1_THOUGHT}
Fitness (MACRO_F1, higher is better): {SCORE}
Task MACRO_F1: {TASK_ID}={TASK_SCORE}
Backend MACRO_F1: {TASK_ID}[random_forest={SCORE}]
Symbolic experience:
{SYMBOLIC_EXPERIENCE_1}
Code:
```python
{PARENT_1_SOURCE}
```

Output only one plain Python snippet: first a `# Thought:` one-sentence comment, then the revised score_split(y_left, y_right, context) function. Do not wrap it
       in a Markdown code fence or add other prose.
````

A.4 Operator Instructions

The operator instruction is the only part of the user message that differs between operators. E1 and E2 recombine three parents, while M1, M2, and M3 modify one.

E1 (explore)

```
Create a useful scoring heuristic with a substantially different mathematical form from the parent heuristics. Preserve only the fixed interface and DSL
    constraints.
```

E2 (recombine)

```
Compare the parent symbolic experiences. Preserve well-supported successful splits and change the purity-versus-balance tradeoff at weak nodes. Create a new
    mathematical form from the observed failure patterns rather than copying a parent.
```

M1 (repair)

```
Study the reported validation errors, weak leaves, node diagnostics, and symbolic tree paths. Refine the parent split score to address one supported failure
    pattern while preserving successful behavior. A report can contain just one CART tree; best and worst representatives can then be the same tree. Do not
    assume unreported ensemble diversity.
```

M2 (tune)

```
Use node and leaf diagnostics to tune the relative weights of candidate-dependent purity, balance, or small-child penalties. Target the declared validation
    objective; do not merely rescale the whole score by a positive constant.
```

M3 (simplify)

```
Use the available tree overview and node diagnostics to identify whether purity gain, partition balance, or split information is over-emphasized. Simplify the
    heuristic around a candidate-dependent tradeoff that addresses observed validation failures.
```

A.5 Symbolic Experience

The symbolic experience in each parent block is not produced by an LLM. It is computed from the parent's fitness forest on the development data and inserted under `Symbolic experience:`. The listing shows the beginning of the experience attached to the best Whisky program in the final generation; the forest overview reports the dominant and unused root features, frequent feature interactions, redundant paths, and the validation samples on which the trees disagree most, followed by the best representative tree with the split scores and node context of every split.

Symbolic experience (Whisky, excerpt)

```
Task whisky_outer7_inner7 (binary classification; 271 train / 91 val):

## Meta information
{"n_cat": 6, "n_num": 0, "n_train": 271, "n_trees": 10, "n_val": 91, "name": "1.0", "num_classes": 2, "num_features": 6, "task_type": "binclass"}

## Forest overview (10 trees, mean val AUROC = 0.7344)
- Dominant root split: "Cost" starts 8/10 trees, "Type" starts 2/10 trees.
- Under-explored (never a root split): "Whisky", "Class", "Cluster", "Country".
- Most repeated interaction pairs along root-to-leaf paths: "C"+"W" (124 paths), "C"+"T" (103 paths), "T"+"W" (56 paths).
- 30 redundant path cluster(s) (Jaccard >= 0.8). Largest (7 paths): the rule "Cost != over 300 CAD and Cost == between 125 and 300 CAD and Cluster == Medium-
    bodied, medium-sweet, fruity, honey, malty, winey and Whisky != Cragganmore NAS (Special Release 2016)" recurs.
- High-disagreement val samples:
    * sample 59: tree votes spread across 4 distinct values (std=0.423).
      - tree0: Cost == over 300 CAD and Cost == between 125 and 300 CAD and Cluster == Medium-bodied, medium-sweet, fruity, honey, malty, winey and Type == Malt
      and Whisky == Willett Family Estate Rye XCF 1.0 and Country == USA -> predict class1 (0.57)
      - tree1: Cost == between 125 and 300 CAD and Cluster == Medium-bodied, medium-sweet, fruity, honey, malty, winey and Country == Japan and Whisky == Spirit
      of Hven Seven Stars No. 3 Phecda and Cluster == Medium-bodied, medium-sweet, smoky, medicinal, spicy, fruity, nutty -> predict class0 (1.00)
      - tree2: Cost == between 125 and 300 CAD and Whisky == Lismore 21yo and Class == Scotch-like -> predict class1 (1.00)
    * sample 62: tree votes spread across 4 distinct values (std=0.423).
      - tree0: Cost == over 300 CAD and Cost == between 125 and 300 CAD and Cluster == Medium-bodied, medium-sweet, fruity, honey, malty, winey and Type == Malt
      and Whisky == Willett Family Estate Rye XCF 1.0 and Country == USA -> predict class1 (0.57)
      - tree1: Cost == between 125 and 300 CAD and Cluster == Medium-bodied, medium-sweet, fruity, honey, malty, winey and Country == Japan and Whisky == Spirit
      of Hven Seven Stars No. 3 Phecda and Cluster == Medium-bodied, medium-sweet, smoky, medicinal, spicy, fruity, nutty -> predict class0 (1.00)
      - tree2: Cost == between 125 and 300 CAD and Whisky == Lismore 21yo and Class == Scotch-like -> predict class1 (1.00)
    * sample 33: tree votes spread across 2 distinct values (std=0.400).
      - tree0: Cost == over 300 CAD -> predict class0 (1.00)
      - tree1: Cost == between 125 and 300 CAD and Cost == over 300 CAD -> predict class0 (1.00)
      - tree2: Cost == between 125 and 300 CAD and Cost == over 300 CAD and Country == Scotland -> predict class0 (1.00)
```

```
    * sample 39: tree votes spread across 2 distinct values (std=0.400).
      - tree0: Cost == over 300 CAD -> predict class0 (1.00)
      - tree1: Cost == between 125 and 300 CAD and Cost == over 300 CAD -> predict class0 (1.00)
      - tree2: Cost == between 125 and 300 CAD and Cost == over 300 CAD and Country == Scotland -> predict class0 (1.00)

## Tree 5 (best tree, val AUROC = 0.7643, +0.030 vs forest)
{
 "feature": "Cost",
 "category": "over 300 CAD",
 "operator": "==",
 "split_score": 0.158905,
 "node_context": {
  "depth": 0,
  "depth_fraction": 0.0,
  "node_sample_count": 271,
  "node_fraction": 1.0,
  "feature_use_count": 0,
[... 969 further lines: rest of the best tree, worst representative tree, and recommendations ...]
```

A.6 Repair Prompt

When a response cannot be parsed or violates the program contract, the error is returned to the LLM in a follow-up user message, at most three times per offspring.

Repair prompt

```
The previous response was invalid ({VALIDATION_ERROR}). Fix the reported error and check the entire DSL contract before returning a different valid program.
      Inline helpers; replace collection item assignments with comprehensions or scalar sums; replace method calls with allowed bare functions. Read documented
      context keys with context["key"] only in node_context mode. Do not add imports, nested def, lambda, methods, range, or keyword arguments. Return only a #
      Thought: one-sentence comment followed by exactly one def score_split(y_left, y_right, context): function, with no Markdown or other prose.
```

B Evolved Split Programs

This section lists the fourteen split-scoring programs used for EvoGraft in Tables 1 and 2, one per dataset. Each program is the best individual of its search, selected by fitness on the development-validation split of the evolution pool. For every program we give the generation and operator that produced it, its development-validation fitness, and the one-sentence rationale (*Thought*) that the LLM returned with the code. The programs are reproduced verbatim.

B.1 Classification Programs

B.1.1 Chocolate. Found in generation 11 by operator M1; development-validation macro-F1 0.699.
Thought: Keep the twoing-plus-gain core and minority rescue that produced the best tree, but harden the path-reuse penalty (especially for features repeating on the same path deep in the tree) and add a redundant-candidate damping term so path-diverse splits outrank re-splits of already-used features, targeting the worst tree's repeated Review_Date branching and the 35 redundant path clusters.

```python
def score_split(y_left, y_right, context):
    n_left = float(len(y_left))
    n_right = float(len(y_right))
    total = n_left + n_right
    if total <= 0.0 or n_left <= 0.0 or n_right <= 0.0:
        return 0.0
    parent = list(y_left) + list(y_right)
    labels = set(parent)
    separation = 0.0
    for label in labels:
        p_left = sum(1 for value in y_left if value == label) / n_left
        p_right = sum(1 for value in y_right if value == label) / n_right
        separation = separation + abs(p_left - p_right)
    separation = separation / 2.0
    size_factor = (n_left / total) * (n_right / total) * 4.0
    twoing = size_factor * separation
    parent_probs = [sum(1 for value in parent if value == label) / total for label in labels]
    parent_gini = 1.0 - sum(probability ** 2 for probability in parent_probs)
    left_gini = 1.0 - sum((sum(1 for value in y_left if value == label) / n_left) ** 2 for label in labels)
    right_gini = 1.0 - sum((sum(1 for value in y_right if value == label) / n_right) ** 2 for label in labels)
    weighted_gini = (n_left * left_gini + n_right * right_gini) / total
    gini_gain = (parent_gini - weighted_gini) / parent_gini if parent_gini > 0.000000000001 else 0.0
    balance = context["balance"]
    depth_fraction = context["depth_fraction"]
    balance_weight = 0.25 + 0.6 * depth_fraction
    core = twoing * (1.0 - balance_weight + balance_weight * balance) + 0.45 * gini_gain * balance
    path_use = float(context["path_feature_use_count"])
    tree_use = float(context["feature_use_count"])
    reuse_penalty = 1.0 / (1.0 + (0.8 + 1.6 * depth_fraction) * path_use + 0.15 * tree_use)
    novelty_bonus = 1.0 + 0.2 * (1.0 - depth_fraction) * (1.0 - separation) if path_use == 0 else 1.0
    min_fraction = min(n_left, n_right) / total
    minority_gain = 0.0
    for label in labels:
        p_left = sum(1 for value in y_left if value == label) / n_left
        p_right = sum(1 for value in y_right if value == label) / n_right
        parent_prob = sum(1 for value in parent if value == label) / total
        child_purity = max(p_left, p_right)
        minority_gain = minority_gain + parent_prob * (child_purity - parent_prob)
    purity_factor = 0.5 + 0.5 * max(0.0, minority_gain)
    soft_support = min(1.0, min_fraction / 0.07)
    minority_boost = 1.0 + 0.12 * (1.0 - min(1.0, min_fraction / 0.2)) if min_fraction < 0.2 else 1.0
    purity_rescue = 1.0 + 0.3 * separation * (1.0 - min(1.0, min_fraction / 0.12)) if min_fraction < 0.12 else 1.0
    high_gain = 1.0 + 0.1 * gini_gain if path_use > 0 else 1.0
    return float(core * reuse_penalty * novelty_bonus * purity_factor * soft_support * minority_boost * purity_rescue * high_gain)
```

B.1.2 Coffee. Found in generation 19 by operator M3; development-validation macro-F1 0.602.
Thought: Validation failures show near-tiny-child categorical splits (balance ~0.02-0.09) winning on marginal purity and 'roaster' repeating on paths, so I simplify to normalized Gini gain gated by a balance exponent that is strict at shallow depths and relaxes with depth, plus a path-reuse and global feature-reuse penalty.

```python
def score_split(y_left, y_right, context):
    parent = list(y_left) + list(y_right)
    labels = set(parent)
    n_left = float(len(y_left))
    n_right = float(len(y_right))
    total = n_left + n_right
```

```
    parent_gini = 0.0
    left_gini = 0.0
    right_gini = 0.0
    for label in labels:
        parent_probability = sum(1 for value in parent if value == label) / total
        left_probability = sum(1 for value in y_left if value == label) / n_left
        right_probability = sum(1 for value in y_right if value == label) / n_right
        parent_gini = parent_gini + parent_probability * parent_probability
        left_gini = left_gini + left_probability * left_probability
        right_gini = right_gini + right_probability * right_probability
    parent_gini = 1.0 - parent_gini
    left_gini = 1.0 - left_gini
    right_gini = 1.0 - right_gini
    if parent_gini <= 0.000000000001:
        return 0.0
    weighted_gini = (n_left * left_gini + n_right * right_gini) / total
    gini_gain = (parent_gini - weighted_gini) / parent_gini
    if gini_gain <= 0.0:
        return 0.0
    balance = context["balance"]
    depth_fraction = context["depth_fraction"]
    balance_exponent = 2.2 - 1.6 * depth_fraction
    balance_factor = balance ** balance_exponent
    path_reuse = float(context["path_feature_use_count"])
    total_reuse = float(context["feature_use_count"])
    reuse_penalty = 0.10 * path_reuse + 0.03 * min(3.0, total_reuse)
    score = gini_gain * balance_factor * (1.0 - reuse_penalty)
    return float(max(0.0, score))
```

B.1.3 Michelin. Found in generation 12 by operator M3; development-validation macro-F1 0.674.
Thought: Validation failures show shallow-depth splits over-weighted by aggressive balance shaping while repeated path features persist, so simplify to normalized MI gain with a mild monotone balance tilt, a stronger super-linear path-reuse penalty, and a shallow-depth novelty bonus to diversify root features.

```
def score_split(y_left, y_right, context):
    n_left = float(len(y_left))
    n_right = float(len(y_right))
    total = n_left + n_right
    if total <= 0.0 or n_left <= 0.0 or n_right <= 0.0:
        return 0.0
    parent = list(y_left) + list(y_right)
    labels = set(parent)
    parent_counts = [sum(1 for value in parent if value == label) for label in labels]
    left_counts = [sum(1 for value in y_left if value == label) for label in labels]
    right_counts = [sum(1 for value in y_right if value == label) for label in labels]
    parent_entropy = 0.0 - sum((float(count) / total) * log(float(count) / total) for count in parent_counts if count > 0)
    left_entropy = 0.0 - sum((float(count) / n_left) * log(float(count) / n_left) for count in left_counts if count > 0)
    right_entropy = 0.0 - sum((float(count) / n_right) * log(float(count) / n_right) for count in right_counts if count > 0)
    conditional_entropy = (n_left / total) * left_entropy + (n_right / total) * right_entropy
    if parent_entropy <= 0.0:
        return 0.0
    gain = parent_entropy - conditional_entropy
    per_bit = gain / parent_entropy
    depth_fraction = context["depth_fraction"]
    balance = context["balance"]
    feature_index = context["feature_index"]
    ancestors = context["ancestor_feature_indices"]
    path_use = float(context["path_feature_use_count"])
    total_use = float(context["feature_use_count"])
    novelty = 1.0 if not any(value == feature_index for value in ancestors) else 0.0
    reuse_factor = 1.0 / (1.0 + 1.5 * path_use * path_use + 0.1 * total_use)
    shallow = clamp(1.0 - 2.0 * depth_fraction, 0.0, 1.0)
    novelty_bonus = 1.0 + (0.20 + 0.50 * shallow) * novelty
    balance_factor = 0.7 + 0.3 * balance
    score = per_bit * reuse_factor * novelty_bonus * balance_factor
    if not (score == score) or score > 1000000.0 or score < -1000000.0:
        return 0.0
    return float(score)
```

B.1.4 Spotify. Found in generation 18 by operator E2; development-validation macro-F1 0.792.

Thought: Blend the entropy-gain core with a saturating minority-impurity term and a depth-shaped balance schedule, add strong shallow path-reuse penalties to break the root monoculture, and modulate the small-child penalty by node support so weak low-support nodes lean on balance.

```python
def score_split(y_left, y_right, context):
    parent = list(y_left) + list(y_right)
    labels = set(parent)
    n_left = float(len(y_left))
    n_right = float(len(y_right))
    total = n_left + n_right
    parent_entropy = 0.0
    left_entropy = 0.0
    right_entropy = 0.0
    for label in labels:
        parent_probability = sum(1 for value in parent if value == label) / total
        left_probability = sum(1 for value in y_left if value == label) / n_left
        right_probability = sum(1 for value in y_right if value == label) / n_right
        if parent_probability > 0.0:
            parent_entropy = parent_entropy - parent_probability * log(parent_probability)
        if left_probability > 0.0:
            left_entropy = left_entropy - left_probability * log(left_probability)
        if right_probability > 0.0:
            right_entropy = right_entropy - right_probability * log(right_probability)
    if parent_entropy <= 0.000000000001:
        return 0.0
    weighted_entropy = (n_left * left_entropy + n_right * right_entropy) / total
    gain = (parent_entropy - weighted_entropy) / parent_entropy
    balance_score = 2.0 * min(n_left, n_right) / total
    depth_fraction = context["depth_fraction"]
    node_fraction = context["node_fraction"]
    path_use = float(context["path_feature_use_count"])
    total_use = float(context["feature_use_count"])
    path_penalty = 1.0 / (1.0 + (1.0 - depth_fraction) * (1.0 - depth_fraction) * 1.3 * path_use)
    use_penalty = 1.0 / (1.0 + 0.1 * total_use * (1.0 - depth_fraction))
    minority_entropy = min(left_entropy, right_entropy)
    minority_factor = 1.0 - minority_entropy / parent_entropy
    minority_boost = 1.0 - minority_factor * minority_factor
    small_child_ratio = min(n_left, n_right) / total
    support = 1.0 - node_fraction
    balance_weight = clamp(0.35 + 0.45 * support - 0.15 * depth_fraction, 0.1, 0.85)
    minority_weight = clamp(0.3 + 0.4 * depth_fraction, 0.1, 0.8)
    base = gain * (1.0 - balance_weight + balance_weight * balance_score)
    shaped = base * (1.0 - minority_weight + minority_weight * (0.5 * minority_factor + 0.5 * minority_boost))
    small_child_penalty = min(small_child_ratio * 5.0, 1.0)
    support_gate = 0.55 + 0.45 * (1.0 - support) + 0.25 * support * small_child_penalty
    return float(max(shaped * path_penalty * use_penalty * support_gate, 0.0))
```

B.1.5 US Accidents. Found in generation 13 by operator M3; development-validation macro-F1 0.676.
Thought: Diagnostics show purity gain and minority-floor shaping are reasonable while the stacked multipliers (margin, diversity, support) over-constrain the search; I simplify to a candidate-dependent tradeoff of normalized gini gain, balance, and minority-floor with a mild path-repeat penalty.

```python
def score_split(y_left, y_right, context):
    parent = list(y_left) + list(y_right)
    labels = set(parent)
    n_left = float(len(y_left))
    n_right = float(len(y_right))
    total = n_left + n_right
    if total <= 0.0 or n_left <= 0.0 or n_right <= 0.0:
        return 0.0
    parent_gini = 1.0 - sum((sum(1 for value in parent if value == label) / total) ** 2 for label in labels)
    if parent_gini <= 1e-12:
        return 0.0
    left_gini = 1.0 - sum((sum(1 for value in y_left if value == label) / n_left) ** 2 for label in labels)
    right_gini = 1.0 - sum((sum(1 for value in y_right if value == label) / n_right) ** 2 for label in labels)
    weighted_gini = (n_left * left_gini + n_right * right_gini) / total
    gain = (parent_gini - weighted_gini) / parent_gini
    minority_floor = 1.0
    for label in labels:
        left_share = sum(1 for value in y_left if value == label) / n_left
        right_share = sum(1 for value in y_right if value == label) / n_right
```

```
        minority_floor = min(minority_floor, max(left_share, right_share))
    balance = context["balance"]
    depth_fraction = context["depth_fraction"]
    balance_weight = 0.7 + 0.5 * depth_fraction
    minority_weight = 1.3 - 0.5 * depth_fraction
    balance_term = (balance_weight * balance + minority_weight * minority_floor) / (balance_weight + minority_weight)
    path_repeats = float(context["path_feature_use_count"])
    use_repeats = float(context["feature_use_count"])
    repeat_penalty = 1.0 / (1.0 + 0.15 * path_repeats + 0.05 * use_repeats)
    score = gain * balance_term * repeat_penalty
    return float(max(0.0, score))
```

B.1.6 Whisky. Found in generation 5 by operator M3; development-validation macro-F1 0.758.
Thought: Diagnostics suggest purity and balance are already well-tuned but the heavy multiplicative reuse penalty over-shrinks legitimate deep re-splits, so simplify to a single normalized gini-entropy purity core scaled by a candidate-dependent depth-modulated balance term and one mild reuse factor.

```
def score_split(y_left, y_right, context):
    parent = list(y_left) + list(y_right)
    labels = set(parent)
    n_left = float(len(y_left))
    n_right = float(len(y_right))
    total = n_left + n_right
    if total <= 0.0:
        return 0.0
    parent_gini = 1.0 - sum((sum(1 for value in parent if value == label) / total) ** 2 for label in labels)
    left_gini = 1.0 - sum((sum(1 for value in y_left if value == label) / n_left) ** 2 for label in labels)
    right_gini = 1.0 - sum((sum(1 for value in y_right if value == label) / n_right) ** 2 for label in labels)
    weighted_gini = (n_left * left_gini + n_right * right_gini) / total
    gini_score = (parent_gini - weighted_gini) / parent_gini if parent_gini > 0.000000000001 else 0.0
    parent_entropy = 0.0
    left_entropy = 0.0
    right_entropy = 0.0
    for label in labels:
        parent_probability = sum(1 for value in parent if value == label) / total
        left_probability = sum(1 for value in y_left if value == label) / n_left
        right_probability = sum(1 for value in y_right if value == label) / n_right
        if parent_probability > 0.0:
            parent_entropy = parent_entropy - parent_probability * log(parent_probability)
        if left_probability > 0.0:
            left_entropy = left_entropy - left_probability * log(left_probability)
        if right_probability > 0.0:
            right_entropy = right_entropy - right_probability * log(right_probability)
    weighted_entropy = (n_left * left_entropy + n_right * right_entropy) / total
    entropy_score = (parent_entropy - weighted_entropy) / parent_entropy if parent_entropy > 0.000000000001 else 0.0
    purity = 0.6 * gini_score + 0.4 * entropy_score
    balance_score = context["balance"]
    depth_fraction = context["depth_fraction"]
    depth_weight = 1.1 - 0.6 * depth_fraction
    if depth_weight < 0.4:
        depth_weight = 0.4
    balance_term = (depth_weight - 0.35) + depth_weight * balance_score
    path_reuse = float(context["path_feature_use_count"])
    total_reuse = float(context["feature_use_count"])
    reuse_term = 1.0 / (1.0 + 0.25 * path_reuse * (0.5 + depth_fraction) + 0.05 * total_reuse)
    score = purity * balance_term * reuse_term
    if score != score or score > 1000000.0:
        return 0.0
    return float(score)
```

B.1.7 Yelp. Found in generation 3 by operator M3; development-validation macro-F1 0.653.
Thought: The parents over-weight balance and redundancy penalties relative to the candidate's own class-distribution signal, so I simplify to a normalized Gini gain core with a candidate-dependent tradeoff that relaxes balance pressure when the split's minority separation is strong, modulated by depth, with only a light path-reuse dampener.

```
def score_split(y_left, y_right, context):
    parent = list(y_left) + list(y_right)
    labels = set(parent)
    n_left = float(len(y_left))
    n_right = float(len(y_right))
```

```
    total = n_left + n_right
    parent_gini = 1.0 - sum((sum(1 for value in parent if value == label) / total) ** 2 for label in labels)
    left_gini = 1.0 - sum((sum(1 for value in y_left if value == label) / n_left) ** 2 for label in labels)
    right_gini = 1.0 - sum((sum(1 for value in y_right if value == label) / n_right) ** 2 for label in labels)
    weighted_gini = (n_left * left_gini + n_right * right_gini) / total
    if parent_gini <= 0.000000000001:
        return 0.0
    gain = (parent_gini - weighted_gini) / parent_gini
    if gain <= 0.0:
        return 0.0
    balance = 2.0 * min(n_left, n_right) / total
    depth_fraction = context["depth_fraction"]
    separation = max((abs(sum(1 for value in y_left if value == label) / n_left - sum(1 for value in y_right if value == label) / n_right) for label in labels)
      )
    balance_weight = (1.0 - depth_fraction) * (0.55 + 0.45 * balance) + depth_fraction * (0.3 + 0.7 * balance)
    separation_weight = 1.0 + depth_fraction * separation
    path_dampener = 1.0 / (1.0 + 0.2 * context["path_feature_use_count"])
    return float(gain * balance_weight * separation_weight * path_dampener)
```

B.2 Regression Programs

B.2.1 Anime Planet. Found in generation 20 by operator M2; development-validation RMSE 0.645.
Thought: Evolve Parent 1 by making the small-child penalty depth-adaptive and variance-aware (penalizing tiny, high-variance children more strongly), while keeping the gain core and weakening tree-wide reuse demotion at depth so deep nodes can still refine with informative features.

```python
def score_split(y_left, y_right, context):
    left = [float(value) for value in y_left]
    right = [float(value) for value in y_right]
    parent = left + right
    n_left = float(len(left))
    n_right = float(len(right))
    total = n_left + n_right
    parent_mean = sum(parent) / total
    left_mean = sum(left) / n_left
    right_mean = sum(right) / n_right
    parent_mse = sum((value - parent_mean) ** 2 for value in parent) / total
    left_mse = sum((value - left_mean) ** 2 for value in left) / n_left
    right_mse = sum((value - right_mean) ** 2 for value in right) / n_right
    weighted_mse = (n_left * left_mse + n_right * right_mse) / total
    if parent_mse > 0.000000000001:
        gain = (parent_mse - weighted_mse) / parent_mse
    else:
        gain = 0.0
    depth_fraction = context["depth_fraction"]
    balance = context["balance"]
    path_use = float(context["path_feature_use_count"])
    tree_use = float(context["feature_use_count"])
    left_fraction = context["left_fraction"]
    right_fraction = context["right_fraction"]
    small_child = min(left_fraction, right_fraction)
    balance_weight = 0.2 * (1.0 - depth_fraction)
    shape = (1.0 - balance_weight) + balance_weight * balance
    reuse_pressure = 0.1 + 0.25 * (1.0 - depth_fraction)
    path_factor = 1.0 / (1.0 + reuse_pressure * path_use)
    tree_factor = 1.0 / (1.0 + 0.05 * tree_use * (0.5 + depth_fraction))
    child_penalty = 1.0 - 0.3 * depth_fraction * (1.0 - small_child) * (1.0 - small_child)
    mean_gap = abs(left_mean - right_mean)
    spread = sqrt(max(left_mse, right_mse) + 0.000000000001)
    risk = spread / (spread + mean_gap + 0.000000000001)
    stability = 1.0 - 0.25 * depth_fraction * (1.0 - small_child) * risk
    adjusted = gain * shape * path_factor * tree_factor * child_penalty * stability
    if adjusted > 1.0:
        adjusted = 1.0
    if adjusted < -1.0:
        adjusted = -1.0
    return float(adjusted)
```

B.2.2 Beer Ratings. Found in generation 10 by operator M2; development-validation RMSE 0.387.
Thought: I keep normalized MSE gain as the core but diversify the forest by penalizing globally over-used and path-repeated features, demanding near-equal balance at deep nodes, and imposing a small-child penalty so near-degenerate partitions must earn their keep through exceptional purity gain.

```python
def score_split(y_left, y_right, context):
    left = [float(value) for value in y_left]
    right = [float(value) for value in y_right]
    parent = left + right
    n_left = float(len(left))
    n_right = float(len(right))
    total = n_left + n_right
    parent_mean = sum(parent) / total
    left_mean = sum(left) / n_left
    right_mean = sum(right) / n_right
    parent_mse = sum((value - parent_mean) ** 2 for value in parent) / total
    left_mse = sum((value - left_mean) ** 2 for value in left) / n_left
    right_mse = sum((value - right_mean) ** 2 for value in right) / n_right
    weighted_mse = (n_left * left_mse + n_right * right_mse) / total
    if parent_mse > 0.000000000001:
        gain = (parent_mse - weighted_mse) / parent_mse
    else:
```

```
        gain = 0.0
    spread = max(parent) - min(parent)
    if spread > 0.000000000001:
        edge_score = (max(left_mean, right_mean) - min(left_mean, right_mean)) / spread
    else:
        edge_score = 0.0
    balance = context["balance"]
    depth_fraction = context["depth_fraction"]
    path_repeat = float(context["path_feature_use_count"])
    tree_repeat = float(context["feature_use_count"])
    candidate_pos = float(context["candidate_index"])
    candidate_total = float(context["candidate_count"])
    path_penalty = 1.0 / (1.0 + 0.45 * path_repeat)
    tree_penalty = 1.0 / (1.0 + 0.12 * tree_repeat)
    depth_weight = 0.3 + 0.7 * depth_fraction
    balance_term = (1.0 - depth_weight) + depth_weight * balance
    if balance < 0.1:
        small_child_penalty = 0.85 + 1.5 * balance
    else:
        small_child_penalty = 1.0
    noise_bonus = 1.0 + 0.02 * (candidate_pos / max(candidate_total, 1.0) - 0.5)
    score = gain * (0.6 + 0.4 * edge_score) * balance_term * path_penalty * tree_penalty * small_child_penalty * noise_bonus
    return float(score)
```

B.2.3 Cardekho. Found in generation 1 by operator E2; development-validation RMSE 0.066.
Thought: Blend normalized MSE gain with a balance term whose weight grows with depth, and penalize candidates whose feature already repeats on the path or tree to break the gear_box/model_year over-commitment and redundant paths.

```
def score_split(y_left, y_right, context):
    left = [float(value) for value in y_left]
    right = [float(value) for value in y_right]
    parent = left + right
    n_left = float(len(left))
    n_right = float(len(right))
    total = n_left + n_right
    parent_mean = sum(parent) / total
    left_mean = sum(left) / n_left
    right_mean = sum(right) / n_right
    parent_mse = sum((value - parent_mean) ** 2 for value in parent) / total
    left_mse = sum((value - left_mean) ** 2 for value in left) / n_left
    right_mse = sum((value - right_mean) ** 2 for value in right) / n_right
    weighted_mse = (n_left * left_mse + n_right * right_mse) / total
    if parent_mse > 1e-12:
        gain = (parent_mse - weighted_mse) / parent_mse
    else:
        gain = 0.0
    balance = context["balance"]
    depth_fraction = context["depth_fraction"]
    node_fraction = context["node_fraction"]
    repetition = float(max(context["path_feature_use_count"], context["feature_use_count"]))
    rep_penalty = 1.0 / (1.0 + 0.25 * repetition)
    balance_weight = 0.15 + 0.35 * depth_fraction
    rarity = 1.0 - node_fraction
    rarity_boost = 1.0 + 0.1 * rarity
    score = (gain * ((1.0 - balance_weight) + balance_weight * balance)) * rep_penalty * rarity_boost
    return float(score)
```

B.2.4 Employee Salaries. Found in generation 14 by operator M2; development-validation RMSE 0.100.
Thought: Keep the Welch/ANOVA hybrid core but strengthen the anti-monopoly novelty pressure with a path-repetition penalty scaled by node support, add a sharper mid-balance rejection at depth, and a small-child variance penalty to reduce redundant deep paths and improve validation RMSE.

```
def score_split(y_left, y_right, context):
    left = [float(value) for value in y_left]
    right = [float(value) for value in y_right]
    parent = left + right
    n_left = float(len(left))
    n_right = float(len(right))
    total = n_left + n_right
    left_mean = sum(left) / n_left
```

```
    right_mean = sum(right) / n_right
    parent_mean = sum(parent) / total
    left_var = variance(left) if len(left) > 1 else 0.0
    right_var = variance(right) if len(right) > 1 else 0.0
    parent_var = variance(parent) if len(parent) > 1 else 0.0
    pooled_se = left_var / n_left + right_var / n_right
    if pooled_se > 1e-12:
        t_stat = abs(left_mean - right_mean) / sqrt(pooled_se)
        gap_term = t_stat / (t_stat + 1.0)
    else:
        gap_term = 0.0
    if parent_var > 1e-12:
        explained = (n_left * (left_mean - parent_mean) ** 2 + n_right * (right_mean - parent_mean) ** 2) / total
        anova_share = explained / parent_var
    else:
        anova_share = 0.0
    depth_fraction = context["depth_fraction"]
    node_fraction = context["node_fraction"]
    balance = context["balance"]
    balance_term = (0.35 + 0.65 * sqrt(balance)) * (1.0 - 0.35 * depth_fraction)
    mid_reject = 1.0 - 0.25 * exp(-40.0 * abs(balance - 0.5)) * depth_fraction
    path_use = float(context["path_feature_use_count"])
    tree_use = float(context["feature_use_count"])
    freshness = 1.0 / (1.0 + log1p(path_use) + 0.1 * log1p(tree_use))
    if path_use == 0.0 and tree_use == 0.0:
        freshness = freshness * 1.2
    repeat_penalty = 1.0 / (1.0 + 0.5 * path_use * node_fraction)
    candidate_index = float(context["candidate_index"])
    candidate_count = float(context["candidate_count"])
    rank_fraction = candidate_index / max(candidate_count, 1.0)
    novelty_gate = 1.0 + (0.15 * rank_fraction) * (1.0 - 2.0 * depth_fraction)
    min_frac = min(n_left, n_right) / total
    support_floor = 1.0 - exp(-14.0 * min_frac)
    small_child_var = (min(n_left, n_right) / total) * (left_var if n_left < n_right else right_var)
    scale = parent_var if parent_var > 1e-12 else 1.0
    small_penalty = 1.0 + 0.3 * (small_child_var / scale)
    core = 0.4 * gap_term + 0.6 * anova_share
    score = core * balance_term * mid_reject * freshness * repeat_penalty * novelty_gate * support_floor * small_penalty
    if context["split_kind"] == "categorical":
        score = score * 1.04
    return float(max(min(score, 1e6), 0.0))
```

B.2.5 Museums. Found in generation 7 by operator E2; development-validation RMSE 0.454.
Thought: I keep normalized MSE gain as the core signal but replace the parents' piecewise sliver penalty and linear-depth reuse weight with a new form: a tanh-saturated novelty discount on reused features, a pooled-sigma effect-size term that breaks ties among near-equal-gain candidates, and a depth-modulated balance coupling that rewards balanced splits only where gain is weak, so well-supported strong splits survive while redundant deep path clusters and slivers are demoted.

```
def score_split(y_left, y_right, context):
    left = [float(value) for value in y_left]
    right = [float(value) for value in y_right]
    parent = left + right
    n_left = float(len(left))
    n_right = float(len(right))
    total = n_left + n_right
    if total <= 0.0 or n_left <= 0.0 or n_right <= 0.0:
        return 0.0
    parent_mean = sum(parent) / total
    left_mean = sum(left) / n_left
    right_mean = sum(right) / n_right
    parent_var = sum((value - parent_mean) ** 2 for value in parent) / total
    if parent_var <= 0.000000000001:
        return 0.0
    left_var = sum((value - left_mean) ** 2 for value in left) / n_left
    right_var = sum((value - right_mean) ** 2 for value in right) / n_right
    weighted_var = (n_left * left_var + n_right * right_var) / total
    gain = (parent_var - weighted_var) / parent_var
    pooled_sigma = sqrt(max(weighted_var, 0.0))
    if pooled_sigma <= 0.000000000001:
        effect = 0.0
    else:
        effect = abs(left_mean - right_mean) / (2.0 * pooled_sigma)
```

```
        if effect > 1.0:
            effect = 1.0
    depth_fraction = context["depth_fraction"]
    balance = context["balance"]
    path_use = float(context["path_feature_use_count"])
    total_use = float(context["feature_use_count"])
    novelty_excess = max(0.0, total_use - path_use)
    novelty = 1.0 - tanh(0.35 * path_use + 0.2 * novelty_excess)
    shallow_bias = 1.0 - depth_fraction
    reuse_factor = 1.0 - (0.45 * shallow_bias + 0.1) * (1.0 - novelty)
    min_share = min(n_left, n_right) / total
    sliver_factor = min(1.0, min_share / 0.08)
    weak_gain = max(0.0, 1.0 - gain)
    balance_coupling = 1.0 + 0.3 * weak_gain * (balance - 0.5)
    score = gain * (0.85 + 0.15 * effect) * reuse_factor * sliver_factor * balance_coupling
    if score < 0.0:
        score = 0.0
    return float(score)
```

B.2.6 Used Cars. Found in generation 3 by operator M3; development-validation RMSE 0.118.
Thought: The base MSE gain is sound but the forest over-reuses dominant features, so I add a candidate-dependent penalty on path and tree-wide feature repetition (stronger near the root where diversity matters most) plus a mild balance preference to suppress redundant path clusters.

```
def score_split(y_left, y_right, context):
    left = [float(value) for value in y_left]
    right = [float(value) for value in y_right]
    parent = left + right
    n_left = float(len(left))
    n_right = float(len(right))
    total = n_left + n_right
    parent_mean = sum(parent) / total
    left_mean = sum(left) / n_left
    right_mean = sum(right) / n_right
    parent_mse = sum((value - parent_mean) ** 2 for value in parent) / total
    left_mse = sum((value - left_mean) ** 2 for value in left) / n_left
    right_mse = sum((value - right_mean) ** 2 for value in right) / n_right
    weighted_mse = (n_left * left_mse + n_right * right_mse) / total
    if parent_mse > 0.000000000001:
        gain = (parent_mse - weighted_mse) / parent_mse
    else:
        gain = 0.0
    balance = context["balance"]
    depth_fraction = context["depth_fraction"]
    path_use = float(context["path_feature_use_count"])
    total_use = float(context["feature_use_count"])
    root_count = float(context["root_sample_count"])
    node_count = float(context["node_sample_count"])
    diversity_weight = 1.0 - depth_fraction
    repetition = path_use * (0.05 + 0.06 * diversity_weight) + 0.015 * total_use
    score = gain * (1.0 - repetition) * (0.85 + 0.15 * balance)
    if root_count > 0.0 and node_count / root_count < 0.02:
        score = score * 0.9
    if score < 0.0:
        score = 0.0
    return float(score)
```

B.2.7 Wine Prices. Found in generation 20 by operator M3; development-validation RMSE 0.730.
Thought: The parent heuristic over-weights MAD/overlap separation terms that add variance without improving ordering; I simplify to a robust variance-reduction core (a stable ordering signal), then apply only candidate-dependent factors that provably reorder splits: a mild balance fit, a small-child penalty, and a stronger path/tree feature-repeat penalty that grows with depth to curb the worst tree's redundant 'Year' re-use and the 45 redundant path clusters.

```
def score_split(y_left, y_right, context):
    left = [float(value) for value in y_left]
    right = [float(value) for value in y_right]
    n_left = float(len(left))
    n_right = float(len(right))
    total = n_left + n_right
```

```
    parent = left + right
    parent_mean = sum(parent) / total
    left_mean = sum(left) / n_left
    right_mean = sum(right) / n_right
    var_left = variance(left)
    var_right = variance(right)
    var_parent = variance(parent)
    if var_parent <= 1e-12:
        return 0.0
    child_var = (n_left * var_left + n_right * var_right) / total
    gain = max(0.0, 1.0 - child_var / var_parent)
    mean_gap = abs(left_mean - right_mean) / (sqrt(var_parent) + 1e-9)
    shift = tanh(mean_gap)
    depth_fraction = context["depth_fraction"]
    balance = context["balance"]
    node_fraction = context["node_fraction"]
    n_features = float(context["n_features"])
    path_use = float(context["path_feature_use_count"])
    tree_use = float(context["feature_use_count"])
    small_child = min(n_left, n_right) / total
    small_penalty = min(1.0, small_child / (0.15 + 0.2 * depth_fraction))
    path_allowance = max(0.5, 1.6 - 0.8 * depth_fraction)
    path_repeat = min(1.0, path_use / path_allowance)
    tree_repeat = min(1.0, tree_use / max(2.0, 0.7 * n_features))
    repeat_pressure = 1.0 - (0.35 * path_repeat + 0.2 * tree_repeat) * (0.5 + 0.5 * depth_fraction)
    if repeat_pressure < 0.2:
        repeat_pressure = 0.2
    ideal_balance = 0.35 + 0.3 * depth_fraction
    span = max(ideal_balance, 1.0 - ideal_balance)
    balance_fit = 1.0 - abs(balance - ideal_balance) / span
    if balance_fit < 0.0:
        balance_fit = 0.0
    support = min(1.0, node_fraction * 4.0 + 0.25)
    quality = 0.7 * gain + 0.3 * shift
    score = quality * balance_fit * repeat_pressure * small_penalty * support
    if score != score or score > 1e12:
        return 0.0
    return float(max(0.0, score))
```

C Selected Correction Structures

Table 2 summarizes the structures that cross-validation selects in each uncertainty bucket of the CARTE runs. Linear correctors are selected most often in classification and tree correctors in regression, while constant correctors are less frequently preferred. Buckets with high disagreement and low leaf support, whose samples pass through thin leaves on which the trees disagree, receive the most expressive correctors: the most features in both tasks and, in regression, the largest share of tree correctors. The bucket with high disagreement and high support, which is also the smallest, more often receives constant or linear correctors with fewer features. The mean trust threshold is high, so most samples of a bucket are used at full weight. This variation suggests that a single corrector structure would not suit all uncertainty regions of the forest.

Table 2: Cross-validated correction structures by disagreement × leaf-support bucket, pooled over seven CARTE datasets per task and ten training runs on a fixed train/test split for each dataset. C/L/T: constant/linear/tree shares (%); $\bar{a}$: mean feature count; $\bar{\tau}$: mean trust threshold.

		Classification			Regression		
Disagr.	Support	C/L/T (%)	$\bar{a}$	$\bar{\tau}$	C/L/T (%)	$\bar{a}$	$\bar{\tau}$
High	Low	3/46/51	3.2	0.95	3/11/86	3.5	0.78
	Mid	6/43/51	2.9	0.89	9/23/69	3.2	0.84
	High	22/59/19	2.1	0.87	23/31/46	2.4	0.79
Mid	Low	20/40/40	2.7	0.85	6/31/63	3.3	0.78
	Mid	12/53/34	3.1	0.85	0/29/71	3.4	0.84
	High	11/49/40	2.7	0.92	3/29/69	3.1	0.87
Low	Low	23/43/33	2.3	0.87	11/26/63	2.7	0.84
	Mid	10/33/57	2.6	0.79	9/29/63	3.1	0.79
	High	23/50/27	2.5	0.83	6/23/71	3.4	0.86
All buckets		14/46/40	2.7	0.87	8/26/67	3.1	0.82

D Additional Results for the Main Experiments

This section complements Tables 1 and 2 with the computational cost of every method and with significance tests. All numbers are computed from the same ten training runs as the two tables, using a fixed train/test split for each dataset.

D.1 Training and Inference Time

Tables 3–6 report wall-clock seconds per run, averaged over the training runs. All runs used one workstation with a 6-core (12-thread) CPU; TabPFN-v2 ran on an NVIDIA Quadro T2000 GPU (4 GB), every other method on the CPU. Several runs shared the machine, so the times indicate orders of magnitude rather than exact costs. The training time of EvoGraft covers fitting the ensemble with the evolved split score and selecting the residual corrector by cross-validation; it excludes the one-time split-program evolution, whose cost is reported in Table 7. The training times of LLM-Tree, FeatLLM and MAPLE include their LLM calls. Our DeLTa wrapper runs the whole DeLTa pipeline (rule-guided forest, refinement and calibration) when predicting, so its training column is close to zero and its inference column contains its training cost; its LLM rules are generated once per dataset and cached.

Table 3: Training time on the CARTE classification tables: wall-clock seconds for fitting on the training split, averaged over the training runs of Table 1.

Dataset	EvoGraft	LLM-Tree	DeLTa	FeatLLM	MAPLE	RF	ET	XGB	LGBM	TabPFN-v2
Chocolate	21.2	25.5	<0.01	524	23.3	2.10	2.79	0.30	0.12	1.99
Coffee	11.8	31.9	<0.01	268	22.7	2.32	3.00	0.36	0.12	1.05
Michelin	33.4	25.5	<0.01	188	23.3	11.4	15.6	0.99	0.25	0.99
Spotify	420	19.0	0.02	134	71.1	141	382	6.03	1.55	2.79
US Accidents	119	21.1	<0.01	215	38.9	53.4	80.9	1.61	0.81	1.51
Whisky	10.4	26.3	<0.01	221	20.7	0.80	0.78	0.12	0.06	1.03
Yelp	404	22.2	<0.01	585	59.0	2.84	2.57	11.0	4.71	1.74
Mean	146	24.5	<0.01	305	37.0	30.6	69.7	2.92	1.09	1.59

Table 4: Inference time on the CARTE classification tables: wall-clock seconds for predicting the corresponding test split, averaged over the training runs of Table 1.

Dataset	EvoGraft	LLM-Tree	DeLTa	FeatLLM	MAPLE	RF	ET	XGB	LGBM	TabPFN-v2
Chocolate	0.26	0.02	10.8	0.31	0.21	0.23	0.23	0.02	0.03	0.47
Coffee	0.22	0.02	12.7	0.53	0.20	0.30	0.30	0.02	0.02	0.39
Michelin	0.68	0.02	12.1	0.29	0.22	1.24	1.21	0.04	0.06	1.09
Spotify	6.39	0.15	96.5	0.37	2.89	13.0	23.5	0.30	0.22	17.8
US Accidents	2.18	0.09	15.8	0.46	0.32	5.32	5.81	0.07	0.11	4.73
Whisky	0.21	0.01	11.6	0.33	0.16	0.09	0.10	<0.01	<0.01	0.31
Yelp	8.77	0.21	96.0	1.26	2.42	0.83	0.86	0.72	0.53	17.2
Mean	2.67	0.07	36.5	0.51	0.92	3.00	4.58	0.17	0.14	6.00

Table 5: Training time on the CARTE regression tables: wall-clock seconds for fitting on the training split, averaged over the training runs of Table 2.

Dataset	EvoGraft	DeLTa	MAPLE	RF	ET	XGB	LGBM	TabPFN-v2
Anime Planet	82.2	<0.01	53.8	19.3	18.8	3.09	0.59	2.70
Beer Ratings	32.5	<0.01	70.1	1.92	1.73	0.81	0.21	3.22
Cardekho	311	<0.01	44.3	22.0	25.0	1.58	1.01	2.13
Employee Sal.	88.5	0.01	42.4	3.44	3.61	0.49	0.25	1.30
Museums	85.8	<0.01	44.2	20.4	21.2	3.93	0.82	2.57
Used Cars	22.5	<0.01	46.7	1.68	1.61	0.45	0.25	1.68
Wine Prices	247	<0.01	25.6	14.2	14.5	1.67	0.36	1.65
Mean	124	<0.01	46.7	11.9	12.4	1.72	0.50	2.18

Table 6: Inference time on the CARTE regression tables: wall-clock seconds for predicting the corresponding test split, averaged over the training runs of Table 2.

Dataset	EvoGraft	DeLTa	MAPLE	RF	ET	XGB	LGBM	TabPFN-v2
Anime Planet	1.23	49.0	0.28	1.84	1.18	0.09	0.14	3.42
Beer Ratings	0.21	47.8	0.82	0.24	0.11	0.03	0.04	1.86
Cardekho	8.06	33.2	0.31	1.84	1.22	0.10	0.13	12.2
Employee Sal.	0.52	36.2	0.52	0.33	0.18	0.03	0.03	1.76
Museums	0.71	41.8	0.59	2.07	1.49	0.11	0.15	4.62
Used Cars	0.23	45.6	0.79	0.16	0.10	0.02	0.03	1.09
Wine Prices	0.93	34.6	0.24	1.45	0.94	0.06	0.09	3.79
Mean	1.70	41.2	0.51	1.13	0.74	0.06	0.09	4.10

D.2 LLM Token Usage

Table 7 reports the LLM usage of the split-program evolution of EvoGraft, which runs once per dataset on the evolution pool; the selected program is frozen and reused in every training run, so prediction makes no LLM calls.

D.3 Statistical Significance

Per dataset. Tables 8 and 9 test, for every dataset, whether EvoGraft differs from each baseline over the training runs. *Across datasets.* Table 10 applies the Friedman test to the per-dataset mean scores, and Table 11 compares EvoGraft with each baseline across datasets with the Wilcoxon signed-rank test, separately per task and pooled over both tasks.

The per-dataset tests describe variation across training runs on a fixed train/test split for each dataset; they do not assess variation over repeated split-program searches. The Friedman test assesses differences among methods as a group; rejecting its null hypothesis does not identify which pairs differ. Pairwise conclusions should use the Holm-corrected tests. The pooled tests summarize relative improvements

Table 7: LLM usage of the one-time split-program evolution behind EvoGraft (one evolution run of 20 generations per dataset, `glm-5.3`): requests, prompt, completion and total tokens, and cumulative LLM call time in hours. The evolved program is then reused in every training run, so this cost is not repeated per run.

Dataset	Requests	Prompt	Completion	Total	LLM time (h)
Chocolate	181	4.30M	147.5K	4.45M	0.81
Coffee	188	2.13M	122.2K	2.25M	0.74
Michelin	185	3.47M	163.5K	3.63M	0.63
Spotify	192	3.26M	148.6K	3.41M	0.68
US Accidents	187	4.39M	144.5K	4.53M	2.39
Whisky	174	2.82M	131.2K	2.95M	0.71
Yelp	183	5.78M	146.3K	5.93M	0.68
Anime Planet	170	3.75M	133.0K	3.89M	1.10
Beer Ratings	171	3.18M	117.7K	3.30M	1.25
Cardekho	173	4.47M	139.1K	4.61M	1.29
Employee Sal.	170	4.19M	131.1K	4.32M	1.29
Museums	172	3.54M	120.1K	3.66M	0.90
Used Cars	172	3.37M	115.9K	3.49M	1.16
Wine Prices	167	3.19M	142.3K	3.33M	1.00
Total	2485	51.86M	1.90M	53.76M	14.63

over the included datasets from both tasks, rather than establishing an advantage within each task separately. A non-significant result does not establish equivalence, particularly with only seven datasets per task.

Table 8: Per-dataset significance on the classification tables: EvoGraft against each baseline, two-sided Mann–Whitney U test using repeated training runs on a fixed train/test split for each dataset, Holm-corrected across the baselines of each dataset. +/−: EvoGraft significantly better/worse at $\alpha = 0.05$; ≈: no significant difference detected. Adjusted p-values in parentheses; W/T/L counts significant wins, non-significant comparisons, and significant losses, with T indicating no detected difference rather than equivalence.

Dataset	LLM-Tree	DeLTa	FeatLLM	MAPLE	RF	ET	XGB	LGBM	TabPFN-v2
Chocolate	+ (.002)	+ (.015)	+ (.002)	≈ (.307)	+ (.002)	+ (.002)	+ (.002)	+ (.002)	+ (.014)
Coffee	+ (.005)	+ (.011)	+ (.034)	+ (.001)	+ (<.001)	+ (.001)	+ (.011)	+ (.034)	+ (.001)
Michelin	+ (.002)	≈ (.689)	+ (.004)	≈ (.312)	≈ (.051)	≈ (.303)	≈ (.127)	≈ (.127)	≈ (.689)
Spotify	+ (.002)	≈ (1.000)	+ (.002)	+ (.002)	+ (.002)	+ (.002)	+ (.002)	+ (.002)	≈ (1.000)
US Accidents	+ (.002)	≈ (.063)	+ (.002)	− (.023)	+ (.002)	+ (.002)	≈ (.811)	≈ (.811)	+ (.002)
Whisky	+ (.002)	+ (.002)	+ (.028)	+ (.011)	+ (.002)	+ (.002)	− (.002)	− (.002)	≈ (.104)
Yelp	+ (.001)	≈ (.345)	+ (.001)	− (.001)	+ (.001)	+ (.001)	+ (.001)	+ (.001)	+ (.006)
W/T/L	7/0/0	3/4/0	7/0/0	3/2/2	6/1/0	6/1/0	4/2/1	4/2/1	4/3/0

Table 9: Per-dataset significance on the regression tables: EvoGraft against each baseline, two-sided Mann–Whitney U test using repeated training runs on a fixed train/test split for each dataset, Holm-corrected across the baselines of each dataset. +/−: EvoGraft significantly better/worse at $\alpha = 0.05$; ≈: no significant difference detected. Adjusted p-values in parentheses; W/T/L counts significant wins, non-significant comparisons, and significant losses, with T indicating no detected difference rather than equivalence.

Dataset	DeLTa	MAPLE	RF	ET	XGB	LGBM	TabPFN-v2
Anime Planet	+ (.001)	≈ (.385)	+ (.001)	+ (.001)	+ (.001)	+ (.001)	≈ (.242)
Beer Ratings	+ (.001)	+ (.001)	+ (.001)	+ (.001)	+ (.001)	+ (.001)	− (.001)
Cardekho	+ (.001)	+ (.001)	+ (.001)	+ (.001)	+ (.001)	+ (.001)	+ (.001)
Employee Sal.	+ (.001)	≈ (.064)	+ (.001)	+ (.001)	+ (.001)	+ (.001)	− (.001)
Museums	+ (.001)	+ (.001)	+ (.001)	+ (.001)	+ (.001)	+ (.001)	+ (.007)
Used Cars	+ (.001)	− (.001)	+ (.001)	+ (.001)	+ (.001)	+ (.001)	+ (.001)
Wine Prices	+ (.001)	≈ (.385)	+ (.001)	+ (.001)	+ (.001)	+ (.001)	− (.001)
W/T/L	7/0/0	3/3/1	7/0/0	7/0/0	7/0/0	7/0/0	3/1/3

Table 10: Friedman test of the null hypothesis that all methods perform equally, on the per-dataset mean scores (RMSE negated so that higher is better). Only methods with results on every dataset of the block enter the test.

Block	Datasets	Methods	χ^2_F	p
Classification	7	10	39.22	<.001
Regression	7	8	36.62	<.001
Both tasks	14	8	56.86	<.001

Table 11: Across-dataset comparison of EvoGraft with each baseline: median relative improvement of EvoGraft over the baseline across datasets (positive = EvoGraft better; macro-F1 gain or RMSE reduction) and two-sided Wilcoxon signed-rank p-value on the per-dataset relative improvements, Holm-corrected across baselines within each block (*: $p < 0.05$). With seven datasets per task the smallest attainable unadjusted p is $1/64 \approx 0.016$; the pooled block uses the fourteen datasets of both tasks for the baselines evaluated on both.

Baseline	Classification (7)		Regression (7)		Both tasks (14)	
	Median	p	Median	p	Median	p
LLM-Tree	+47.6%	.141	–	–	–	–
DeLTa	+4.0%	.547	+8.5%	.109	+7.1%	.004*
FeatLLM	+19.0%	.141	–	–	–	–
MAPLE	+0.8%	.688	+2.1%	1.000	+1.1%	.535
RF	+12.7%	.141	+29.0%	.109	+25.4%	<.001*
ET	+12.7%	.141	+30.2%	.109	+26.2%	<.001*
XGB	+1.1%	.547	+11.1%	.109	+6.7%	.004*
LGBM	+1.0%	.547	+10.8%	.109	+6.5%	.004*
TabPFN-v2	+2.0%	.547	-2.0%	1.000	+1.0%	.535